

Master Thesis

**Vantage-Point Tree to HNSW: Fast Index Construction for
Approximate Nearest Neighbor Search on High-Dimensional
Data**

Tadeo Hepperle
236400

February 13, 2025

Begutachtung:
Prof. Dr. Erich Schubert
Andreas Lang

Technische Universität Dortmund
Fakultät für Informatik
Data Mining Arbeitsgruppe
<https://dm.cs.tu-dortmund.de/>

Abstract

Search graphs can be used to perform approximate k -NN search on large datasets. Hierarchical Navigable Small Worlds (HNSW) [MY18] and Relative Nearest Neighbor Descent (RNN-Descent) [OM23] are graph construction methods that yield low search times and a high recall. We propose a new *VP-tree ensemble* technique for fast search graph building that combines ideas from HNSW-indices and RNN-Descent with the space partitioning qualities of vantage point trees (VP-tree) [Yia93]. We developed multithreaded Rust implementations of these VP-tree ensembles, HNSW-indices, and RNN-Descent. In k -NN search benchmarks on a high-dimensional vector dataset ($n = 10M$), the VP-tree ensembles built graphs much ($\approx 4x$) faster than the popular *Hnswlib* [MY+18] and *Faiss* [Dou+19] HNSW libraries while sacrificing some recall and search time. Integrating RNN-Descent into parts of the VP-tree ensemble method, yielded both, low build times and high recall.

Contents

1	Introduction	1
2	Basics	5
2.1	Nearest Neighbor Search	5
2.2	Distance Functions	6
2.3	Curse of Dimensionality	6
2.4	Graphs and Greedy Search	7
3	Related Works	11
3.1	Vantage Point Trees	11
3.1.1	Constructing a VP-tree	11
3.1.2	Search in a VP-tree	13
3.2	NN-Descent and Relative NN-Descent	14
3.3	Hierarchical Navigable Small Worlds	18
3.3.1	HNSW Construction and Search	19
4	Methods	23
4.1	Proposed Method	23
4.1.1	Chunk Stitching	25
4.1.2	VP-Tree Ensemble	28
4.2	Benchmark Data	30
4.3	Implementation	31
4.3.1	Construction Parameters	32
4.3.2	Rust Implementation Details	33
5	Results	37
5.1	HNSW	38
5.2	Relative NN-Descent	40
5.3	Chunk Stitching	42
5.4	VP-Tree Ensemble	46
5.5	Benchmarks on the 10M dataset	50
6	Conclusion	59
	Bibliography	61

1 Introduction

Nearest neighbor search (NNS) is a querying problem where, given a query point q , the objective is to find its nearest neighbor in a dataset S . More generally, k -NN search refers to the task of finding the k nearest neighbors of q in S . What constitutes a *point* and which points are considered to be *near* each other depends on the problem domain. For example, points could be digital images, medical records, geographical coordinates, or products in an online store.

A distance function can describe how near points are to each other, but there are many choices for such a function. For coordinates on a map, the distance between two points could be measured via airline or via the path lengths of the roads connecting them. Should we measure the distance in terms of travel time or as a physical distance? Are images of differently colored frogs less similar to each other than two photos of a green frog and a green leaf because both consist of predominantly green pixels?

Answering these philosophical questions should be left to subject-matter experts. Instead, this thesis focuses on a simplified version of k -NN search, where points are just vectors in the d -dimensional vector space $\mathbb{R}^d$, and a distance function $\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is defined. Fortunately, many real-world objects can be mapped to a d -dimensional vector, such that the methods we explore can be applied without loss of generality.

There is a broad field of applications for k -NN search. Recommendation systems can use it to present users with products similar to their previous purchases. Doctors can use k -NN search for searching through medical databases to find cases that are similar to the records of a patient at hand. Search engines utilize k -NN search when searching for images similar to an input image uploaded by a user. In recent years, k -NN search has also been combined with neural networks. For example, Wang, Hamza, and Song [WHS17] describe a method for text classification that passes the k nearest neighbors of a document as additional input to a neural network. On the other hand, neural networks can be used to output embedding vectors that can be mapped to the closest data points in a real dataset with k -NN search. For instance, the *LAION-5B* dataset contains high-dimensional embedding vectors of image-text pairs [Sch+22]. If a user wants to find images of "a white teapot," a text encoder can map this phrase into an embedding vector. Using k -NN search, images from the *LAION-5B* dataset with a similar embedding, likely showing the desired object, can be retrieved.

The simplest way to find the k nearest neighbors to a query point q in a dataset S is a linear scan that calculates the distance between q and every point in S . But this approach suffers from linear search time complexity and can be very slow on large datasets. Sometimes, space partitioning schemes like *k-dimensional trees* (KD-trees), *vantage point trees* (VP-trees) [Yia93] or *quadtrees* [Sam84] can be used to search for nearest neighbors more efficiently. KD-Trees split the data recursively on a single alternating dimension, while quadtrees perform splits on all dimensions at once, forming multiple subtrees. VP-Trees partition a space into subtrees by splitting the data according to their distance to randomly

chosen points, so-called *vantage points*. Quadtrees, for instance, are used for collision detection in 2- or 3-dimensional physics simulations to find objects that are close to each other and might collide [VCC98]. Due to some effects labeled under the term *Curse of Dimensionality*, these tree structures do not work well for k -NN search on high-dimensional data. Search in a VP-tree, for example, relies on spatial pruning that can rarely take effect in higher dimensions.

It is not always required that the neighbors returned by a search are 100% of the true neighbors. Approximate k -NN search (k -ANN search) can be much faster but might return some points that are not among the true k nearest neighbors of the query point. The accuracy of an approximate search is reported by its *recall*, which is defined as the fraction of returned points that are part of the true k nearest neighbors.

An approximate search can be conducted on proximity graphs using an algorithm called *greedy search*. Proximity graphs connect each point in a dataset to neighbors that are near it. Greedy search is a k -ANN search algorithm that starts at any number of entry points in the graph and moves along graph edges towards the query point q . During this process, the best k neighbors are kept in a list and returned when no points closer to q are found anymore among the neighbors of visited points. A graph that contains edges between each point and its k nearest neighbors is called a k -NN graph.

Constructing the exact k -NN graph is expensive because it requires distance calculations between all pairs of points. Therefore, Dong, Moses, and Li [DML11] propose *Nearest Neighbor Descent* (NN-Descent) as a method that approximates the nearest neighbor graph in an iterative fashion. It first connects points at random and then improves these connections locally by connecting a point to the neighbors of its neighbors. Toussaint [Tou80] proposes a structure called *Relative Neighborhood Graphs* (RNG) that is better suited for search than a k -NN graph. In such a graph, clusters are better connected to each other, even if there is a large distance between them. Ono and Matsui [OM23] invented the *Relative NN-Descent* (RNN-Descent) algorithm, which constructs a graph similarly to NN-Descent but approximates the RNG instead of the k -NN graph.

Navigable small worlds (NSW) by Malkov et al. [Mal+12] is another graph structure that aims to speed up greedy search. An NSW graph contains a combination of short and long connections, allowing for long jumps towards a query point, while not sacrificing local accuracy. This idea has been taken up by Malkov and Yashunin [MY18] by building multi-layer graphs called *Hierarchical Navigable Small Worlds* (HNSW). In an HNSW, long connections are found on sparse higher layers, while the lowest layer contains mainly short edges. HNSW structures have been demonstrated fast k -ANN search times and high recall [TAC23]. However, HNSW indices can take a long time to build, because they sequentially insert each individual point into the graph structure.

In this master thesis, we investigate new graph construction algorithms that create search graphs in a comparatively short time. Our new methods are required to work on large datasets and high-dimensional data. While a low construction time is the main objective, we still want to achieve competitive recall and search times. Ideas from the *HNSW*, *Relative NN-Descent*, and *VP-Tree* methods are combined in new ways to achieve this goal. A high-performance Rust implementation was developed to demonstrate the suitability of the new methods on real-world data.

We structure the thesis as follows: In the first chapter, some basics of the nearest neighbor search topic are explained. It covers different distance functions, the curse of dimension-

ality, and k – ANN search on graphs via greedy search. The next chapter describes some related works, building the foundation of our proposed methods. They include *Vantage Point Trees* (VP-Trees) [Yia93], *Hierarchical Navigable Small Worlds* (HNSW) [MY18], and graph construction by *Relative NN-Descent* [OM23]. The *Methods* chapter covers the *Chunk Stitching* and *VP-Tree Ensemble* algorithms we use to tackle the goal of the thesis. It also goes over some details about our implementation in the Rust programming language and the data used for benchmarking.

We compare benchmarks of various search indices in the *Results* chapter, focusing on their build times, search times, and recall. In particular, we analyze how well the different methods were able to build search graphs in parallel. Finally, we draw conclusions from the benchmarking results in the last chapter and discuss additional ideas that could be investigated in the future.

2 Basics

This chapter goes over some basic terminology, math and algorithms that build the foundations of the following chapters.

2.1 Nearest Neighbor Search

Nearest neighbor search (NNS) is a querying task, where given a set of points S in a space M and a query point $q \in M$, the point in S closest to q shall be found. If, instead of a single point, the k closest points in S should be returned, it is called the k -NN search problem. A distance function $M \times M \rightarrow \mathbb{R}$ specifies how close two points are to each other. The distance function depends on the problem domain. In different domains, a "point" can refer to different objects. Points could be relatively unstructured data: nested JSON objects, text segments with variable lengths, or rows in a csv file with possibly missing values. Generally however, objects can be mapped to a d -dimensional vector in $\mathbb{R}^d$. This simplifies the NN-search problem and makes it possible to talk about it in a more general way. The choice of d might come naturally. For example there could be one dimension for every column in a tabular dataset. In other cases it is more tricky to get the data objects mapped into $\mathbb{R}^d$. Textual data, for instance, can be mapped into this vector space by training an attention-based encoder [Vas+23]. We only concern ourselves with the simplified version of the NN-search problem, where points are considered to be d -dimensional vectors in $\mathbb{R}^d$ and a distance function $\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is specified upfront. Not all problems require the neighbors returned by a k -NN search algorithm to be 100% accurate. Relaxing this constraint turns the problem into an *approximate nearest neighbor search* (ANNS). Sacrificing accuracy opens the door to more performant search algorithms.

We now define what ANN search means in the context of this thesis: Let S be a set of distinct points $x_1, \dots, x_n \in \mathbb{R}^d$ and d a distance function $\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$. Assume that the points in S are drawn independently and identically distributed from a distribution H . Let $search$ be a nearest neighbor search algorithm. It can map any point $q \in \mathbb{R}^d$ to a subset $N_{found} \subseteq S$ where $|N_{found}| \leq k \in \mathbb{N}^+$. For any given $k \in \mathbb{N}^+$ and $q \in \mathbb{R}^d$ we can define the k -nearest-neighbors of q as $N_{true} \subseteq S$ with $|N_{true}| = k$ and for any two points $t \in N_{true}$ and $s \in S \setminus N_{true}$ it holds that $d(t, q) \leq d(s, q)$. For any given point q we can calculate a recall value as:

$$recall_{search}(q) = \frac{|search(q) \cap N_{true}|}{|N_{true}|}$$

The recall value is always between 0 and 1. Let now $Q \sim H$ be a random variable from the same distribution as the points in S . We define that the $search$ algorithm performs k -*t*-approximate nearest neighbor search on the dataset S , if and only if $\mathbb{E}[recall_{search}(Q)] \geq t$. For example a method performs 95%-approximate k -NN search, if on average at least 95% of the neighbors returned by this method are part of the true k nearest neighbors.

2.2 Distance Functions

As we just saw, the question of which points are considered to be a part of the true nearest neighbors depends on a concrete distance function. There are different distance functions for different use cases. Consider two vectors $a = a_1, \dots, a_d$ and $b = b_1, \dots, b_d$ in the d -dimensional vector space $\mathbb{R}^d$. The *Euclidean distance*, also known as *L2-distance* is defined as:

$$d_{L2}(a, b) = \|a - b\| = \sqrt{\sum_{i=1}^d (a_i - b_i)^2}$$

The Euclidean distance is symmetrical, such that $d_{L2}(a, b) = d_{L2}(b, a)$, and it is a *norm*. Norms are functions on vector spaces that fulfil certain requirements, among others a property known as the *triangle inequality*: For any 3 vectors a , b and c it holds that $d_{L2}(a, c) \leq d_{L2}(a, b) + d_{L2}(b, c)$ [HKP11, p. 73]. This property is used to prune the search-space in tree-based structures such as KD-trees and VP-trees [Yia93].

A distance measure can also be obtained by inverting a similarity measure. The *cosine similarity* is commonly used to compare embedding vectors in natural language processing. It is defined as [HKP11, p. 78]:

$$\text{sim}_{\cos}(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|} \quad \text{where} \quad a \cdot b = \sqrt{\sum_{i=1}^d a_i b_i}$$

It is equivalent to the cosine of the angle θ between the vectors a and b , thus ranging from -1 (rather dissimilar) to 1 (very similar). A distance measure called *cosine distance* can be defined as $d_{\cos}(a, b) = 1 - \text{sim}_{\cos}(a, b)$. It is not a norm and does not fulfill the triangle inequality. Since it refers to the angle between two vectors it has a special property: for all $x \in \mathbb{R}$ it holds that $d_{\cos}(a, b) = d_{\cos}(a, x \cdot b)$. So scaling a vector by a scalar, does not change its cosine distance to another vector. Vectors for machine learning tasks are often normalized $a_{\text{normalized}} = \frac{a}{\|a\|}$, which implies $\|a_{\text{normalized}}\| = 1$. A dataset consisting of normalized vectors is also referred to as *spherical data*, since all elements lie on the unit (hyper-)sphere. For this kind of data, it holds that $\text{sim}_{\cos}(a, b) = a \cdot b$ because $\|a\| = \|b\| = 1$. Computing just the dot-product is considerably cheaper than calculating the lengths of the individual vectors as well.

2.3 Curse of Dimensionality

When working with high dimensional data, a few challenges arise that can be described under the umbrella term *Curse of Dimensionality*. The most obvious implication of high dimensional spaces is, that algorithms and data structures, that have exponential space or time complexity with respect to the number of dimensions, are not viable. *Quadtrees* for instance, split the space into half on each dimension, such that each tree node contains 2^k pointers to potential subtrees [Sam84]. For 2 and 3 dimensions, each node needs 4 and 8 memory cells respectively, to point to its children. However, for e.g. 768-dimensional data,

one would need more memory cells than there are atoms in the observable universe. Almost the entire space is empty for any high-dimensional dataset, the data becomes *sparse*. In high dimensions, the distribution of space in a hypersphere also changes: The volume of such a sphere is pushed towards a narrow region close to the sphere's surface. The volume V_r of a d -dimensional hypersphere with radius r is given by [DLM24]:

$$V_r = \frac{\pi^{\frac{1}{2}n} r^d}{\Gamma(\frac{1}{2}n + 1)}$$

For any $\epsilon \in (0, 1]$ and $d \in \mathbb{N}^+$ we can calculate, what fraction p of points inside the d -dimensional unit sphere falls into an outer rim of width ϵ (having a maximum distance of ϵ to the sphere's surface). Using the formula above, this fraction can be calculated as $p = 1 - \frac{V_{1-\epsilon}}{V_1} = 1 - (1-\epsilon)^d$. Since $1-\epsilon < 1$ for any $\epsilon \in (0, 1]$, the term $(1-\epsilon)^d$ converges towards 0 for increasing dimensionality d . Thus, p converges towards 1. An example should illustrate this fact. Let us use $\epsilon = 0.01$. In two-dimensional space, $p = 1 - 0.99^2 \approx 0.0199$. So approximately 2% of points in a unit circle are located within a 0.01 distance to its perimeter. If we look at a 768-dimensional hypersphere instead, $p = 1 - 0.99^{768} = 0.9996$, so almost 100% of the volume is located very close to the sphere's surface. This has implications for structures like VP-trees [Yia93], that want to prune the search space by using margins of a certain width.

In the context of nearest neighbors search, the distribution of points also has implications on the distribution of distances between them. As the number of dimensions grows, the distance to the nearest neighbor approaches the distance of the furthest neighbor. Beyer et al. [Bey+99] point out, that this phenomenon makes the notion of a nearest neighbor less meaningful in high dimensions. Due to all distances (near and far) converging towards the same value, small inaccuracies can be devastating to the accuracy of search methods. This is especially the case, considering that most data is encoded using floating point numbers that are susceptible to exactness issues if many math operations are carried out and the margin for error is so slim in high dimensions.

2.4 Graphs and Greedy Search

Search graphs can be used to perform k -ANN search. A graph consists of a set of points S and a set of edges $E \subseteq S \times S$. An undirected graph views edges as unordered tuples, while in a directed graph, the existence of an edge $a \rightarrow b$ does not imply that the reverse edge $b \rightarrow a$ is contained as well. Most graphs encountered in this thesis are considered directed graphs, even though they contain many bidirectional connections in practice. A graph is called a *proximity graph* if it connects points that are close to each other with respect to a distance measure $d : S \times S \rightarrow \mathbb{R}$ [MM19, p. 214].

One type of proximity graph is the k -NN graph. It contains an edge from every point $a \in S$ to each of its k nearest neighbors. Because building a k -NN graph can be costly, often only an approximate version of it is built [OM23].

k -ANN search can be performed on proximity graphs by an algorithm called *Greedy Search*, also known as *Greedy Routing* [MY18]. Pseudocode for it is provided in Algorithm 1.

Algorithm 1 Greedy Search (See *SEARCH-LAYER* in Malkov and Yashunin [MY18])

Input: query point q , list of entry points $ep \subset S$ with $(|ep| \leq ef)$, number of nearest elements to q to return ef , distance function $S \times S \rightarrow \mathbb{R}$

Output: nearest elements R to q with $|R| \leq ef$

```

 $V \leftarrow ep$  ▷ set of visited elements
 $C \leftarrow \{(p, \text{distance}(p, q)) \text{ for } p \in ep\}$  ▷ candidates frontier for breadth-first-search
 $R \leftarrow C$  ▷ priority queue of nearest elements found
while  $|C| > 0$  do
   $c, c_{dist} \leftarrow$  remove the nearest element from  $C$  to  $q$ 
   $f_{dist} \leftarrow$  distance of furthest element in  $R$  to  $q$ 
  if  $c_{dist} > f_{dist}$  then break
  for  $e \in$  neighbors of  $c$  where  $e \notin S$  do
     $S \leftarrow S \cup e$ 
     $f_{dist} \leftarrow$  distance of furthest element in  $R$  to  $q$ 
     $e_{dist} \leftarrow \text{distance}(e, q)$ 
    if  $e_{dist} < f_{dist}$  or  $|R| < ef$  then
      if  $|R| = ef$  then remove furthest element from  $R$ .
       $C \leftarrow C \cup (e, e_{dist})$ 
       $R \leftarrow R \cup (e, e_{dist})$ 

return  $R$ 

```

The algorithm starts with a set of entry points ep and traverses the graph towards a query point q . The search fills a priority queue R with the closest points and their distances to q , as they are encountered [Wan+13, p. 4]. This queue has a maximum capacity of ef elements. There is also a frontier list of candidates C , initialized to the entry points ep . At each step of the search, the candidate $p \in C$ closest to q is removed from this frontier. If the R queue is full ($|R| = ef$) and p has a greater distance to q than each element in the queue, the search terminates.

The search also terminates should C become empty, such that no candidate can be removed from it anymore. Otherwise, all neighbors of p have their distance to q calculated. If the queue R is not full yet or the neighbor is closer to q than the farthest element in the queue, the neighbor is added to the queue and also to the frontier. The algorithm also keeps a set of visited points V , to avoid calculating the distance to any point more than once, and to not get caught in loops jumping back to points that have been already visited.

The search is called greedy, because it only makes locally optimal decisions about which points to add to the candidates. This makes greedy search prone to run into local optima. Running into such a "dead-end" can be somewhat mitigated by increasing the ef parameter and thus including more candidates in general. This works, due to the termination criteria being hit later, for greater ef . For example when $k = 30$ neighbors should be found, it can be a good idea to execute the greedy search with $ef = 100$, and afterwards select the k best elements from the ef returned points.

A k -NN graph does not necessarily make for a good search graph for a given dataset. If the data has spots of different density, greedy search on a k -NN graph can run into local optima quickly.

There can also be regions in a k -NN graph that are not connected to each other at all. Consider the graphs in Figure 2.1, showing connections to the 8 nearest neighbors on 3 different randomly generated datasets.

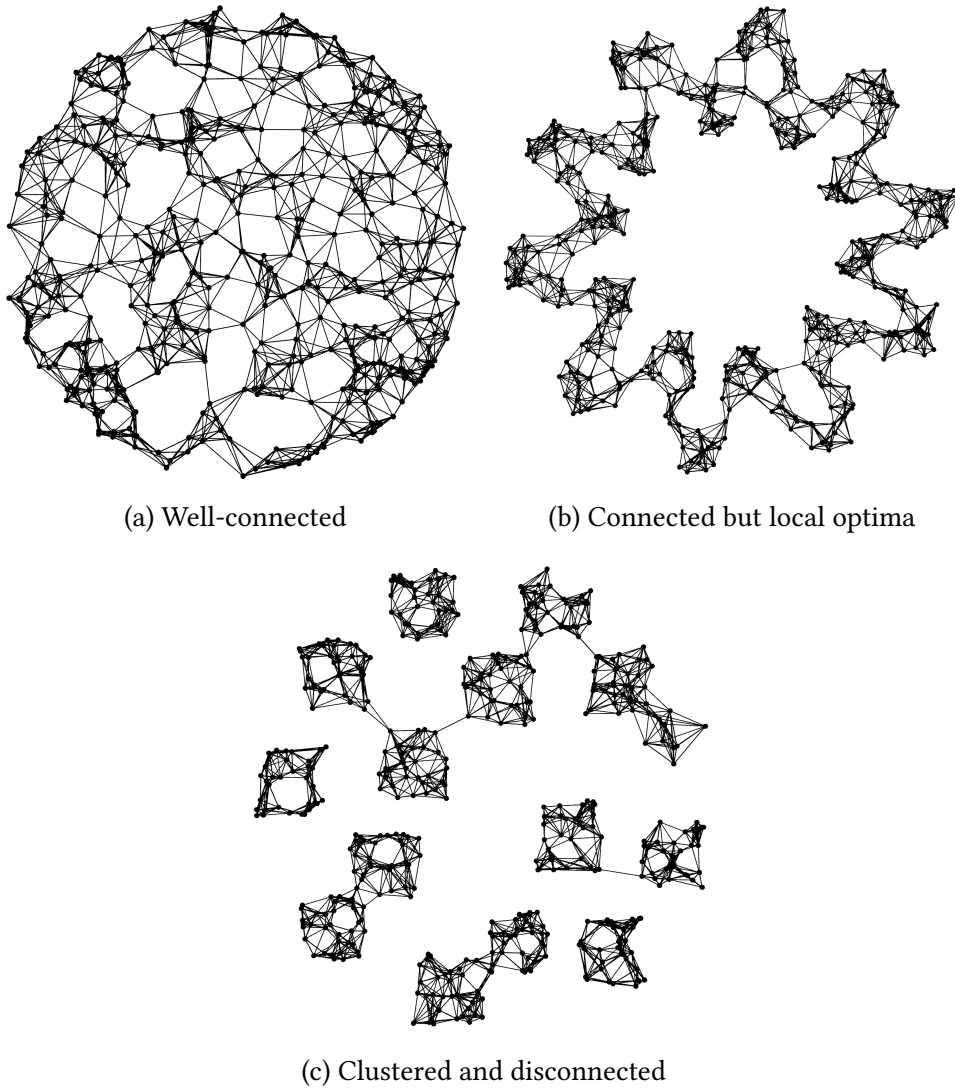


Figure 2.1: Exact k -NN graphs constructed with $k = 8$ on 2-dimensional randomly generated data with $n = 500$ elements, following different distributions.

In Figure 2.1b all points are connected to each other, but the search is likely to get stuck in one of the crevices when searching towards a point from the opposite side of the circle. In Figure 2.1c the data is highly clustered. Here, the k -NN graph fails to connect data points that are too far apart, so that a greedy search never leaves the boundaries of a single cluster. Pure k -NN graphs can also be slow to search because many tiny jumps could be needed to finally converge on a neighborhood close to q .

To make graphs better suited for search, the use of long-range links [Kle00], a layered hierarchy [MY18], or local neighborhood optimizations [Wan+13; OM23; Tou80] have been proposed.

3 Related Works

In this chapter, three classes of search indices are described in detail: VP-trees [Yia93], NN-descent graphs [DML11; OM23] and HNSW-indices [MY18]. For all presented indices, a dataset S consisting of n elements $x_1, \dots, x_n \in \mathbb{R}^d$ is assumed. Furthermore, we assume the use of the Euclidean distance function $d(a, b) = \|a - b\|$, especially when discussing the shape of space regions for a more intuitive understanding. Most of the presented methods generalize well to other distance functions though.

3.1 Vantage Point Trees

A vantage point tree (VP-tree) is a binary search tree. It consists of n nodes that each represent a unique element $x_i \in S$. Nodes point at up to two child nodes, representing a left and a right subtree. The Figure shows arrows from parents to child nodes. A subtree is made up of a node and all of its descendants. There is a root node whose subtree encompasses the entire dataset S . In a VP-tree, every node is called a vantage point and subdivides its subtree into two halves, that are similar in size. All elements in the left subtree of a node representing x_i are closer to the point x_i than all nodes in its right subtree. A point a is closer to x_i than a point b if $d(x_i, a) < d(x_i, b)$. Typically, the Euclidean distance is used here, because search in a VP-tree relies on the distance function being a *norm*.

3.1.1 Constructing a VP-tree

As described by Yianilos [Yia93], a VP-tree containing the points $x_1, \dots, x_n \in S$ is constructed by randomly choosing a vantage point x_i . This point is stored in the root node. The remaining points are sorted by their distance to x_i . Splitting this sequence in the middle yields the elements that make up the left and right subtree respectively. The smallest distance between x_i and a node of the right subtree is stored as a threshold value t in the node x_i . All nodes in the left subtree have a distance to x_i less than t . The subtrees can now be divided recursively, picking a vantage point among all the points the subtree spans, sorting the points by distance to this vantage point and making another split in the middle. The recursion comes to a halt at nodes that do not span at least 2 descendants. For a node with n_{dsc} descendants, n_{dsc} distance calculations between parent node and each of the descendants need to be carried out, to sort descendants into the left and right subtree. Sorting algorithms like *Quicksort* run with $O(n_{dsc} \log(n_{dsc}))$ time complexity [Hoa62]. But sorting the descendants into a left and right subtree does not require a full sort. It is sufficient to create two partitions of approximately equal size, where all elements in the right partition have a higher distance to the parent node than all elements in the left partition. It does not matter how the elements in each partition are ordered, because they will be

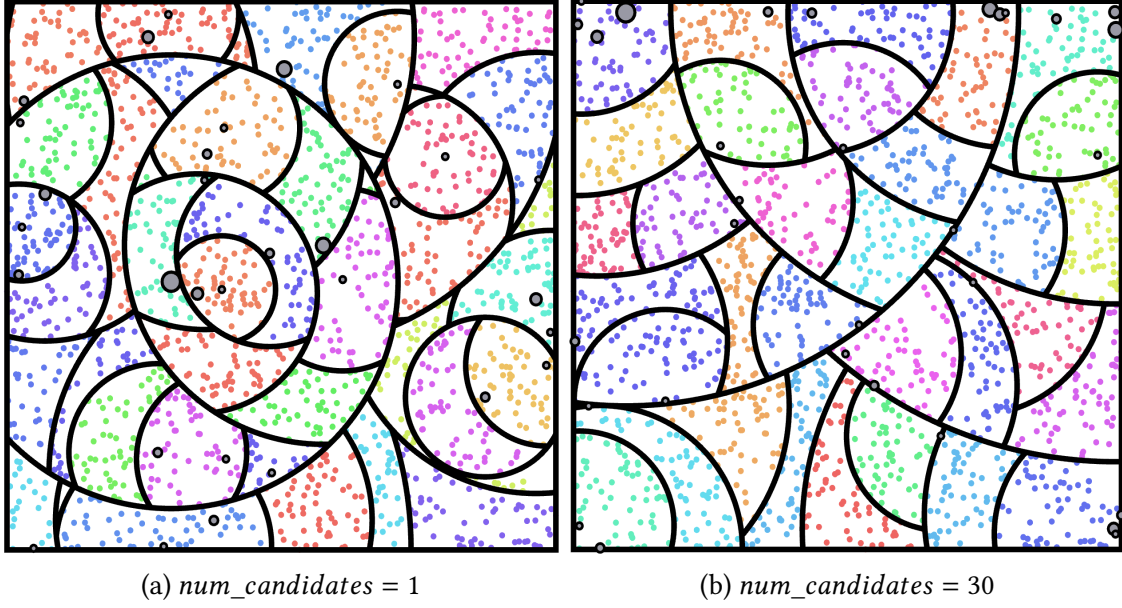


Figure 3.1: Space partitioning by VP-trees built on 2000 random points $\sim U([0, 1]^2)$. Vantage points up to depth 5 are displayed as gray outlined points of decreasing size. Elements that share a subtree at depth 5 are shown in the same color.

reordered in the next recursive split anyway. Using the *Quickselect* algorithm by Hoare [Hoa61], this can be done in $O(n_{dsc})$ time. Constructing the subtrees of each subsequent layer of a VP-tree has the same asymptotic time complexity $O(n)$ as the current layer, because $O(n) = 2 \cdot O((n-1)/2)$. A VP-tree with depth t (root node depth = 0) can hold a maximum of $\sum_{i=1}^{t+1} i^2 = 2^{t+1} - 1$ nodes. Conversely, the depth of a tree with $n > 2$ nodes is $\lfloor \log_2(n) \rfloor$. Thus, a VP-tree can be constructed in $O(n \log n)$ time. It requires $O(n)$ storage space, because there is exactly one node for every element in the dataset. Figure 3.1 shows the structure in which a vantage point tree partitions the space, revealing a pattern of spherical intersections.

When constructing a VP-tree, it might not be the best strategy to pick vantage points randomly. Points differ in how they divide the space. In particular, the threshold distance t , that splits the two subtrees apart and is stored alongside the vantage point, is different for different points. Yianilos [Yia93] notes that it is desired to prefer vantage points with greater t . This means that the distance between elements from two subtrees is more meaningful on average. Because of this, it is argued that points in the corner of a space make for better vantage points. This principle can be applied when constructing a VP-tree. At every node, $n_candidates$ vantage point candidates are sampled instead of a single one. Then, the candidate with the greatest median distance to the other sampled points is chosen. This median distance is a rough estimate for the threshold value t . Instead of the median, the mean can also be used as a heuristic. Sampling multiple candidates for each vantage point slightly increases the construction time, but can yield trees that are better structured for search [Yia93]. Placing a vantage point in the corner of a space can also decrease the "surface area" between the subtrees, which can be important if the tree is used as a sort of clustering. Figure 3.1 shows two VP-trees constructed on the same 2-dimensional random

dataset. The tree built for Figure 3.1a used only one candidate, while the VP-tree in Figure 3.1b sampled 30 candidates and picked the one with the highest mean distance to the other candidates. Random vantage points were chosen for subtrees with less than 30 points in them. Looking at the grey points in the graphics, we see that sampling multiple candidates pushes the vantage points into the corners of the space in their respective subtree. This causes the resulting space partitions to also be slightly more convex.

3.1.2 Search in a VP-tree

A VP-tree can be used to find the k nearest neighbors of a point q . During the search process we keep a candidates list C of the k best points found so far. Initially this list is empty. The size of C is limited to at most k points. If an element should be added to C and C already contains k points, the insertion only succeeds if the element has a lower distance to q than the worst element in C , and can thus replace it. The search is recursive and starts at the root node. Each step considers a node storing a vantage point v and threshold t . We calculate the distance $d(v, q)$ and try to add the node to the candidates list. If $d(v, q) < t$, we search the left subtree by continuing the search with v and t now referring to the left child-node, otherwise the right subtree is searched. After the respective subtree is searched, we can potentially prune the search space and exit the search early. Otherwise, the other subtree needs to be searched as well.

The search can be early-exited, when the *triangle inequality* tells us, that it is impossible for a better candidate to exist in the unsearched space of the other subtree. Suppose after searching the first subtree, C contains k candidates and x_{wc} is the worst of them. Worst means having the greatest distance to q . The triangle inequality states that $d(v, x_{wc}) \leq d(v, q) + d(q, x_{wc})$. In other words, $d(v, q) + d(q, x_{wc})$ is an upper bound for the distance between the vantage point and any point x that could be part of the k -nearest neighbors, requiring $d(x, q) < d(x_{wc}, q)$. This means, if we searched the left (inner) subtree first and $d(v, q) + d(q, x_{wc}) < t$, we know that the right (outer) subtree does not need to be searched anymore, since for every element x in the right subtree it holds that $d(v, x) \geq t$. On the other hand, the triangle inequality also states that $d(v, q) \leq d(v, x_{wc}) + d(x_{wc}, q)$ which can be transformed into $d(v, x_{wc}) \geq d(v, q) - d(x_{wc}, q)$. This tells us, that $d(v, q) - d(x_{wc}, q)$ is a lower bound for the distance $d(v, x)$ between the vantage point and any point x that would be closer to q than x_{wc} . If we searched the right (outer) subtree first and noticed that $d(v, q) - d(x_{wc}, q) > t$, there is no point in considering any of the points in the left subtree anymore, because for all of them it holds that $d(v, x) < t$, by the definition of the VP-tree. Therefore, the search space can be efficiently pruned in either of the two mentioned cases, saving distance comparisons. Notice, that pruning the search in the inner and outer subtree, both rely effectively on a distance to the surface of a hypersphere around the vantage point v . This can be problematic in higher dimensions. As explained above in the section on *Curse of Dimensionality*, most points in a hypersphere are very close to the rim. So in high dimensions pruning can rarely take place while performing k -NN search on a VP-tree. Figure 3.2 shows the fraction of points that were visited while searching for random queries in VP-trees constructed on data with different dimensions. For $d > 20$ virtually all points are visited making the search potentially slower than even a linear search.

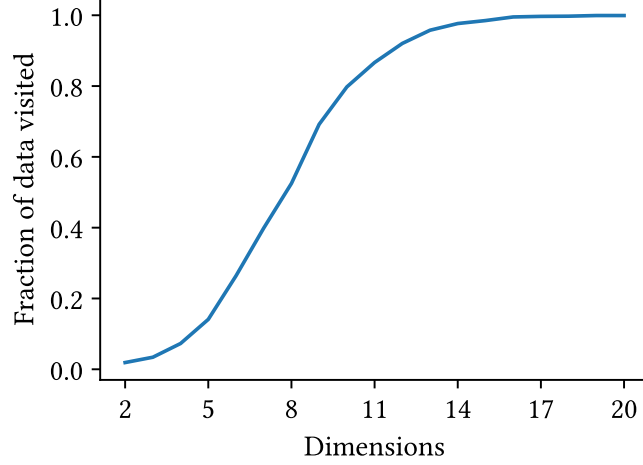


Figure 3.2: Curse of dimensionality impacting the exact NN-search in a VP-tree: In higher dimensions every point is visited during the search computing a distance to the query. Due to branching overhead, this is even less efficient than linear search. For each dimension d a uniformly distributed set of $n = 100000$ points in $[0.0, 1.0]^d$ was generated. Percentages averaged across 1000 queries and 10 unique VP-trees per dimension.

3.2 NN-Descent and Relative NN-Descent

Nearest-Neighbor-Decent (NN-Descent) is a method for building approximate k-NN graphs presented by Dong, Moses, and Li [DML11]. It borrows ideas from gradient descent, a popular method for training neural networks [Jam+13, p. 434], that performs small local improvements along a gradient to update the weights and biases of a neural network. NN-descent follows a similar approach, by improving an approximate NN-graph in small steps. In every step it considers local neighborhoods around points, and optimizes their connections. The idea is, that a neighbor of a neighbor also likely is a neighbor. The algorithm is initialized by creating a random graph in which each point $v \in S$ is assigned a neighbors list B_v by choosing k random other points from S . Then there are multiple descent iterations, that try to optimize the neighbors. The following happens in each iteration: For each $v \in S$, a list $\bar{B}_v$ is created, that consists of the neighbors B_v of v and $\{u \mid v \in B_u\} \forall u \in S$ being all points that v is a neighbor of. These are also called the reverse neighbors of v . Afterwards, for each v we look at each neighbor $u_1 \in \bar{B}_v$ and each of its subsequent neighbors $u_2 \in \bar{B}_{u_1}$. If $d(v, u_2)$ is less than any of distances to v 's original neighbors in B_v , the element with the greatest distance is removed from B_v and u_2 is included in B_v instead. Because a "worse" neighbor is never included in the neighbors lists, the neighborhood graph only gets better in terms of distance between neighbors. NN-Descent can return after a fixed number of iterations or when no more changes occur to the graph. On its own, the approach is not very efficient, because in every iteration each node is compared with k^2 other nodes.

Many of these distance calculations are also repeated in every iteration. There are a few optimizations. Firstly, instead of comparing each node to all neighbors of neighbors, we can compare each neighbor of a node to each other neighbor of that node. In a sense, a

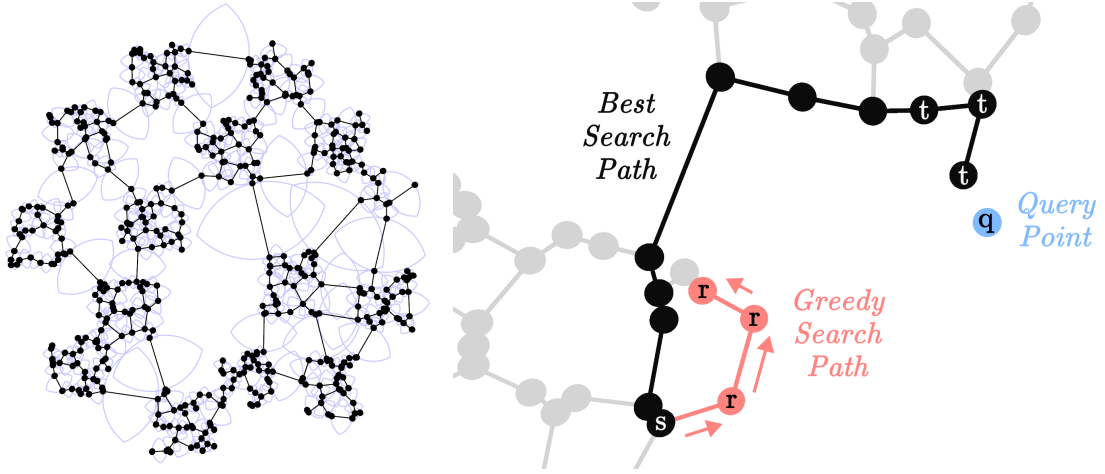
node introduces its neighbors to each other. Dong, Moses, and Li [DML11] call this *Local Join*. It results in $O(k^2)$ distance calculations, but the memory locality is higher. Instead of loading the data for k^2 nodes, now only the k neighboring nodes need to be loaded into memory, in each step of the loop. Secondly, Dong, Moses, and Li [DML11] store a boolean *is_new* flag alongside each point in the neighbors lists. It is initially set to *true* whenever a neighbor is newly inserted into the list. In a *Local Join*, only pairs of neighbors with at least one flag set to *is_new = true* are compared to other neighbors, setting the flags to *is_new = false*. In this way, if a neighborhood has already converged to neighbors that stay relatively stable across iterations, few to none distance calculations are performed. Only a small bookkeeping cost, for checking the flags, remains. It can still happen, that a pair of points is present in the neighbors lists of two other points in the same iteration. In this case they would be introduced to each other twice in a *Local Join*. To combat this, it is proposed to only sample a fraction of neighbors with the *is_new = true* flag in each local join. Dong, Moses, and Li [DML11] show that a low *sample rate* decreases overall recall, but for a *sample rate* > 0.5 this effect seems to be negligible while providing a speed-up to the algorithm.

As mentioned before, a k-NN graph is not necessarily a great search graph to find nearest neighbors via *Greedy Search*, see examples in Figure 2.1. To solve this problem Ono and Matsui [OM23] propose a graph construction method called *Relative NN-Descent* (RNN-Descent) that combines the NN-descent idea with a graph structure called *Relative Neighborhood Graph* (RNG), that is better suited for search. Toussaint [Tou80] defines the RNG as a graph of points where any two points a and b are connected to each other, if there is no other point c , that is closer to a and b than a and b are to each other. In other words, two points $a, b \in S$ are defined as *relatively close* if:

$$d(a, b) \leq \max(d(a, c), d(b, c)) \quad \forall c \in S$$

If the RNG is represented by a directed graph, the graph should contain both edges $a \rightarrow b$ and $b \rightarrow a$ for any *relatively close* points. There are a few differences between the RNG and a k-NN graph. Consider the relative neighborhood graph in Figure 3.3a that is built on the same clustered 2-dimensional data ($n = 500$) as the k-NN graph in Figure 2.1c. Compared to the k-NN graph, the RNG is much sparser. This can be explained by the criterion stated above. For any connection between two points a and b , it essentially defines an intersection area of two circles with radius $d(a, b)$ and centers in a and b . There are no other graph points allowed in this area, because every location in it is at distance $\leq d(a, b)$ from either a or b . A point in this area would violate the RNG property, such that the connection between a and b would be illegal. Figure 3.3a draws these areas for each connection demonstrating that there are indeed no other points found in any of them.

An RNG guarantees connectivity between all nodes. [Tou80] show that there are no areas that are inaccessible from any entry point in the graph. The connectivity follows from the recursive use of the RNG property. The property demands that a connection between two nodes a and b be made (no matter their distance), *if and only if* there are *no points* in the lens-shaped area between a and b . Let us call this *case 1*. In the other case, there is *at least one point* c in this area between a and b . Let us call this *case 2*. In case 2, the RNG rule can now be applied recursively, demanding that there be a direct connection from a to c and from c to b , if the respective circle intersection areas between them do not contain other



(a) Connectivity in an RNG (b) A part of the RNG where greedy search would get stuck

Figure 3.3: Structure of a relative neighborhood graph built on the same $n = 500$ dataset as the k -NN graph in 2.1c. The RNG property defines a shape of two intersecting circles for each connection. This area is void of any other data points, which can you can verify by looking at the area outlined in blue, drawn for every connection in Figure 2.1c. Figure 3.3b shows a subsection of the RNG. It displays the search path (red) of a greedy search with $k = ef = 3$ from point s to q . Points marked with r are returned. Points marked with t are the true 3 nearest neighbors to q and the black path shows the optimal search path towards them.

points. Otherwise, another recursive split is made for connections where this area rule is violated. In any of the two cases a and b are connected to each other, either directly, or via c . This proves the connectivity of any two points in a relative neighborhood graph. Even though there is general connectivity, a greedy search can still get stuck in a local optimum. Figure 3.3b illustrates this, zooming into a subregion (left-to-center) of the graph in Figure 3.3a.

Ono and Matsui [OM23] use this RNG property in their *Relative NN-Descent* algorithm together with an incremental NN-descent-like approach to prune the number of connections and keep connectivity. This leads to an approximation of the RNG instead of an approximation of a k -NN graph.

The *Relative NN-Descent* algorithm by Ono and Matsui [OM23] first assigns a number of m_{initial} random neighbors B_v to each node v , similar to NN-Descent. After this, the algorithm updates the graph in two nested loops with a fixed number of iterations t_{outer} and t_{inner} , see Algorithm 2. Each of the inner loops executes an operation called *UpdateNeighbors* to connect neighbors of each node to each other, if and only if, they fulfill a local RNG criterion. At the end of each iteration of the outer loop (excluding the last one) an *AddReverseEdges* operation is carried out. This adds reverse edges (e.g. $b \rightarrow a$ for $a \rightarrow b$) to the neighbor lists and afterwards prunes the number of connections such that the number of connections *to* and *from* every node each do not exceed a parameter m_{pruned} . The *UpdateNeighbors* method iterates over each point v in the graph. For any pair of distinct neighbors $n_1, n_2 \in B_v$ where $d(v, n_1) < d(v, n_2)$ we check if $d(v, n_2) \geq d(n_1, n_2)$ is fulfilled.

Algorithm 2 Relative NN-Descent [OM23]

Input: dataset S , number of outer loops t_outer , number of inner loops t_inner , initial number of neighbors per node $m_initial$, number of neighbors per node after adding reverse edges m_pruned

Output: neighbor lists $B = B_v \forall v \in S$

```

for  $v \in S$  do
   $B_v \leftarrow$  initialize with  $m\_initial$  random neighbors  $\triangleright is\_new = true$  for each neighbor
  for  $t_1 = 1, \dots, t\_outer$  do
    for  $t_2 = 1, \dots, t\_inner$  do
       $UpdateNeighbors(S, B)$ 
    if  $t_1 \neq t\_outer$  then
       $AddReverseEdges(S, B, m\_pruned)$ 
return  $B$ 

```

If this is the case, and because we know that $(v, n_1) < d(v, n_2)$, we know that the point n_1 lies within the lens-shaped forbidden area between v and n_2 . This violates the RNG area rule explained above. Thus, whenever such a point is encountered, the longer connection $v \rightarrow n_2$ is removed and a connection $n_1 \rightarrow n_2$ is inserted instead. This makes sure that n_2 is still reachable from v on the path $v \rightarrow n_1 \rightarrow n_2$. This path never has a turn sharper than 90 degrees due to the lens shape of the area containing v_1 . If the more distant point v_2 is removed from the neighbors list of v in this way, neighbor pairs where v_2 would be the point closer to v are not considered anymore in this local join. Just like in NN-descent an *is_new* flag is used to skip distance calculations for pairs of neighbors that have both been present in a previous call to *UpdateNeighbors*.

In the *AddReverseEdges* method first each point $v \in S$ has its neighbors list B_v extended by its reverse neighbors $\{u \in S \mid v \in B_u\}$. Since the *UpdateNeighbors* method only enforces the RNG property locally in a neighborhood and not globally like in an exact RNG (as in Figure 3.3a), according to Ono and Matsui [OM23], adding reverse edges back into the graph, is necessary to prevent the graph from converging to a local optimum early on. Afterwards only the top m_pruned shortest incoming edges and the top m_pruned shortest outgoing edges are kept for each point, discarding the rest. This limits the in- and out-degree of each node to a maximum of m_pruned which prevents bloating the graph. Note that, the lens shaped "illegal" areas might look like they keep the graph sparse in 2 dimensions but in higher dimensions they shrink down in proportion to the entire space, likely not providing much pruning. Ono and Matsui [OM23] claim that Relative NN-descent outperforms NN-descent in terms of search performance, recall and construction time on various datasets.

There are a few appealing things about NN-descent and Relative NN-descent. The methods work with a variety of distance functions and their runtime and space complexity does not depend on the dimensionality of the data. They also parallelize well, because work can be done in multiple neighborhoods at the same time with only a small local effect radius. Lists of neighbors need to be synchronized, but the probability for lock contention is low with a high enough *dataset-size-to-number-of-threads* ratio. The graph-building times of these iterative methods can also be easily tuned by changing the number of iterations.

3.3 Hierarchical Navigable Small Worlds

The structure of k-NN graphs and RNGs is not ideal if the greedy search should traverse the graph as fast as possible. Making many small jumps becomes a problem, especially when *a)* distance calculations become expensive for high dimensional data or *b)* the number of neighbors at each visited point is high. To solve this problem *navigable small worlds* (NSW) have been proposed by Malkov et al. [Mal+12]. A *small world* is a network in which any node can reach any other node by a small number of jumps. It was originally introduced by Milgram [Mil67] in the context of social networks. He proposed, that every person in the world be connected to any other person by only six degrees of separation.

A navigable graph is a search graph in which the number of jumps needed to get from one node to another scales logarithmically or polylogarithmically with the size of the dataset [BKC08]. To achieve this scaling, NSW methods insert a combination of short links (i.e. nearest neighbors) and long links into the graph. Long links provide opportunities to make long jumps into some direction cutting down on search time. Short links are necessary to precisely find the closest neighbor in some neighborhood.

In a small world graph, the search process can be split into two phases: "zoom-out" and "zoom-in" [Kle00]. Starting at a random point, the "zoom-out" part of the search jumps to neighbors with higher and higher out-degree, until a maximum is reached. These nodes are called *hubs* and provide a better starting point for the "zoom-in" greedy search (see Algorithm 1) searching towards the query point. A problem is, that the algorithm can get stuck in a local optimum in each of the two phases. It would be faster and more accurate to skip the "zoom-out" phase and start the search directly at one of the hubs.

Malkov and Yashunin [MY18] propose a *Hierarchical Navigable Small World (HNSW)* structure to solve this problem. An HNSW consists of multiple layers of proximity graphs. Higher layers are more sparse, and thus feature longer connections on average, while nodes in lower layers have shorter connections to other nodes due to a higher density. Each node in a higher layer is also present at the layer below and links to it. Figure 3.4 shows an HNSW visualization with points on different layers and connections between them. Like in a NSW-graph, the combination of long and short links leads to better search performance, while the layered nature allows search to always start at the highest layer, going directly into the "zoom-in" phase.

In its layered structure, an HNSW is similar to a skip list. A skip list is a probabilistic data structure consisting of many layers of sorted linked lists. Each element is inserted on a randomly chosen level l and all layers below [Pug90]. The level is chosen from a probability distribution $P[l]$ that results in a constant factor $p = \frac{P[l \geq L]}{P[l \geq L-1]} \forall L \in N^+$ with $p < 1$. The bottom layer is level 0 and contains all n elements. So on average, each layer at level $l > 1$ contains p times as many points as the layer below. The search for an element can then start at the highest level, "skipping" over big areas of the data stored on a lower level of the skip list. The search progresses until a pair of nodes has been found, which marks an interval containing the query. Now the search can jump to the linked list a layer below, narrowing down the search interval further until the element is either found at any layer or level 0 is reached, and the element was not found. This allows for $O(\log n)$ average search time complexity.

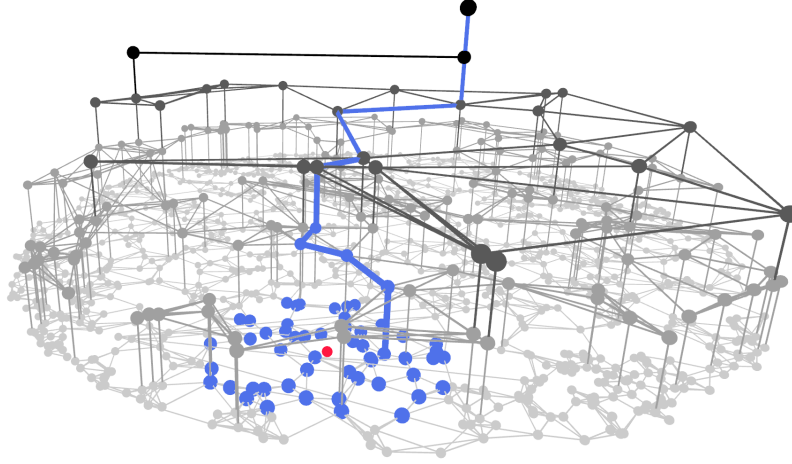


Figure 3.4: An HNSW with $ef_construction = 16$, $m_max = 4$, $m_max_0 = 6$, and $level_norm = 0.5$ built on a 2-dimensional, randomly generated dataset ($n = 1000$). Higher layers are colored in a darker shade. The red point marks a query point and the blue lines are the search path that is taken through the HNSW. The blue points on the bottom layer represent the returned nearest neighbors (with $recall = 100\%$ in this example).

3.3.1 HNSW Construction and Search

An HNSW is constructed and searched similarly to a skip list. Initially, all layers are empty. Each point q in a dataset S is individually inserted into the structure one by one. When inserting an element, a random level $l_{insert} = \lfloor -\ln(U[0, 1]) \cdot level_norm \rfloor$ is chosen. $level_norm$ is a parameter that can be used to control the height of the structure. Each point q is then inserted on each level from 0 to l_{insert} and connected to other nodes on the respective layers. See Algorithm 3 for a detailed look at the insertion process. The insertion starts at an entry point which is the first element of the top layer l_{top} of the HNSW. From there, we perform a greedy search in each layer with $ef = 1$ towards point q (See Algorithm 1). This returns a new entry point for the layer below. We do this until the l_{insert} layer is reached.

From here on, we also perform a greedy search towards q on each layer but use the parameter $ef = ef_construction$ instead. $ef_construction$ should be greater than the other parameters M , m_max and m_max_0 , because the greedy search returns at most ef elements. A new node is inserted for q on the layer and the best M of the found points are declared its neighbors. Each of these neighbors is also connected to q in the reverse direction. We do not want neighbors to have too many connections, so only the best m_max of them are kept. No neighbor list can contain more than m_max elements. On layer zero, a special parameter m_max_0 is used instead. Doing this allows for a finer knitted net on the bottom layer, increasing the chance of hitting the correct neighbors when layer zero is reached during the search. If $l_{insert} > l_{top}$, additional layers are added to the top of the HNSW to make up for the difference, and q becomes the HNSW's new entry point. At that point, every layer added above l_{insert} only contains q with an empty set of neighbors, because no other points exist on these higher layers yet.

Algorithm 3 HNSW Insert (See *INSERT* in Malkov and Yashunin [MY18])

Input: list of graph *layers*, new element q , number of picked neighbors M , maximum number of neighbors m_max_0 (bottom layer) and m_max (higher layers), size of greedy search frontier $ef_construction$, normalization factor for insertion level generation $level_norm$

Output: one or multiple *layers* contain q with connections to and from other points

```

 $l_{insert} \leftarrow \lfloor -\ln(U[0, 1]) \cdot level\_norm \rfloor$ 
 $l_{top} \leftarrow |layers|$ 
 $ep \leftarrow \{ \text{first element in } layers[l_{top}] \}$  ▷ search results and next entry points
for  $l \in l_{top} \dots (l_{top})$  do
     $ep \leftarrow GreedySearch(layers[l], q, ep, ef = 1)$  ▷ returns single element set
for  $l \in \min(l_{top}, l_{insert}) \dots 0$  do
     $layer = layers[l]$ 
     $ep \leftarrow GreedySearch(layer, q, ep, ef = ef_{Construction})$ 
     $neighbors \leftarrow \text{select } M \text{ best neighbors from } ep$ 
     $layer[q] = neighbors$ 
     $M_{max} \leftarrow \text{if } l \neq 0 \text{ then } m\_max \text{ else } m\_max\_0$ 
    for  $e \in neighbors$  do
         $new\_neighbors = layer[e] \cup q$ 
        if  $|new\_neighbors| > M_{max}$  then
             $new\_neighbors \leftarrow \text{select } M_{max} \text{ best neighbors from } new\_neighbors$ 
         $layer[e] = new\_neighbors$ 
if  $l_{insert} > l_{top}$  then
    for  $l \in (l_{top} + 1) \dots l_{insert}$  do
         $layers[l] \leftarrow \text{new layer containing only element } q \text{ with empty neighbors}$ 

```

Two times, Algorithm 3 refers to selecting the *best* M neighbors out of some candidates list C . But what does this mean? Malkov and Yashunin [MY18] propose two different methods. The *simple* method merely selects the M nearest elements in C to the query point q . The *heuristic* method has the goal to establish more connections in diverse directions. It builds and returns a set R with $|R| \leq M$ neighbors in it. R is initially empty. The candidate points in C are iterated in order of their distance to q , with closer points being visited first. A candidate c is only inserted into R if $d(c, q) \leq d(c, r)$ for all $r \in R$.

This is extremely similar to the RNG property discussed before. Essentially each point that is close to q creates a lens-shaped no-go area and all points falling into it are omitted. Malkov and Yashunin [MY18] claim this may increase recall for clustered data because it excludes multiple neighbors that point in the same direction (e.g. the cluster center). Doing so leaves more space in the neighbors list for neighbors that are further away in a different direction (e.g. located in a different cluster). If in the end $|R| < M$, some close points that violate the RNG property can be added back in, to fill up R .

In the 2023 SISAP indexing challenge, HNSW indices performed best among several other methods, reaching low search times and high recall [TAC23]. However, it is important to tune the build parameters according to the data. For example, if a *level_norm* parameter of 0.0 is chosen, the HSNW is completely flat. Malkov and Yashunin [MY18] found

that *level_norm* values between 0.1 and 0.6 produced the best search times depending on the dataset and propose to set it to $\frac{1}{\ln(M)}$. In higher dimensions and for bigger datasets *ef_construction* and *m_max* have to be increased. Regarding the number of neighbors on the bottom layer they found that $m_max_0 = 2 \cdot m_max$ works best. HNSW construction and search works for many distance functions and does not rely on e.g. the triangle inequality.

Searching the k nearest neighbors of a query point q in an HNSW follows a similar pattern as the construction algorithm. We start with an entry point in the top layer. A greedy search with $ef = 1$ searches towards q in that layer. The nearest point found is used as an entry point in the next lower layer and the search towards q repeats. Similar to the search in a skip list, we get closer and closer to q while the density of the graph increases, keeping the workload per layer roughly equal. Once level 0 is reached, a greedy search with $ef = ef_{search}$ is performed in the bottom layer using the last search result of the layer above as the entry point. The k nearest points to q in the search results are returned in the end.

4 Methods

We aim to combine ideas from HNSW [MY18], Relative NN-Descent [OM23], and VP-trees [Yia93], to develop new graph construction methods. Attainment of this goal is measured by three key characteristics: search recall, search time, and index construction time (also referred to as build time). Our main objective is to reduce the build time while still being able to cross a fixed recall threshold of at least 80%. A competitive search duration is also important, but the main focus of our proposed methods is to minimize the build time in comparison to the popular way of constructing an HNSW. To achieve these goals, multiple methods are proposed and have been implemented in the Rust programming language [MK14]. The newly developed methods are benchmarked against existing HNSW implementations. We also developed single- and multithreaded reference implementations for a VP-tree, an HNSW-index, and Relative NN-Descent, in Rust. A Python package exposing the functionality of the underlying Rust library has also been created to simplify benchmarks against existing HNSW implementations.

4.1 Proposed Method

Our proposed methods use the space partition properties of a VP-Tree to break down the graph construction problem into small chunks. Connections are made within the chunks before they are merged into a global graph covering the dataset. We propose two different methods, for this merge operation: *Chunk stitching* and *VP-tree ensembles*. Let us start, by taking a closer look at the structure of a VP-tree.

Subtrees of a VP-tree at a fixed depth have almost the same size (± 1). They partition the space into distinct, (mostly) connected regions as visible in Figure 3.1. We want to use these subtrees to form chunks that contain points with at least a few close neighbors in the same chunk. A *chunk* is an unordered set of points from the dataset. We assign a unique identifier I to each chunk, making it easier to talk about its relation to other chunks in the context of a VP-tree. This identifier is an ordered list of $n \geq 0$ plus and minus symbols $(I_1, I_2, \dots, I_n)$, such that $I_i \in \{+, -\}$. A chunk always stands for a subtree of the VP-tree. The chunk with the empty identifier $I = ()$ covers all elements of the dataset. Let the chunk with identifier $I = (+)$ contain the top vantage point of the VP-tree and all points in its left (positive) subtree. Conversely, the chunk with $I = (-)$ covers the rest of the data, which is the right (negative) subtree under the root node. The identifier represents the path from the root node to a subtree. Every *plus* sign navigates into the left (positive) subtree, while a *minus* sign leads to the right (negative) half. Thus, any subtree at depth n has a list of n symbols as its identifier I and can be split into two chunks $(I_1, \dots, I_n, +)$ and $(I_1, \dots, I_n, -)$. An important detail is that we consider the vantage point of a node to always be a part of the left chunk. To keep all chunks at a fixed depth similar in size (± 1), we make a tiny modification to the usual VP-tree median split rule: When the number of elements that

need to be split at their median distance to a vantage point is uneven, the right (negative) subtree receives one more element than the left subtree. If the number of elements is even, the negative subtree gets 2 more elements than the positive subtree. As a result, the size of the left chunk (containing the left subtree and one or many parent vantage points) is always the same size or 1 less than the right chunk.

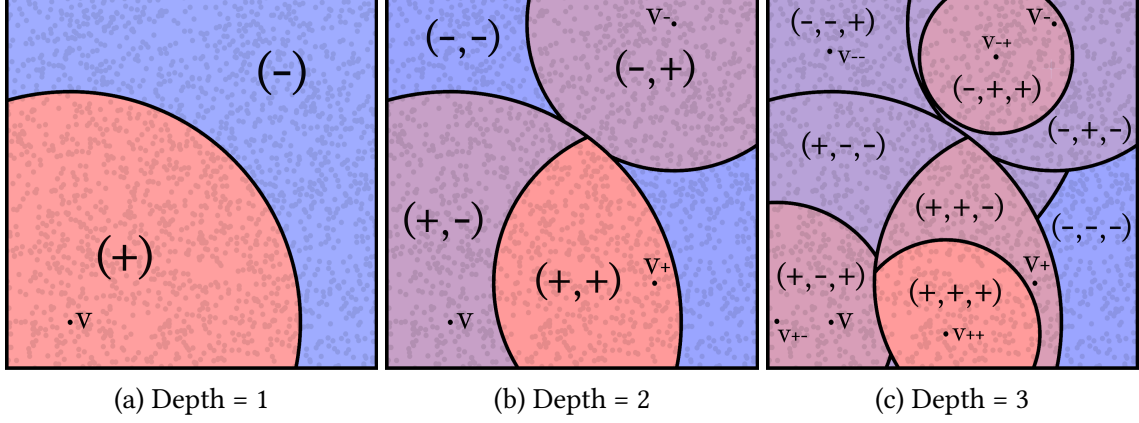


Figure 4.1: Chunk identifiers for a VP-tree built on 2-dimensional data

Figure 4.1 visualizes chunks and their identifiers at different depths. The vantage points are also shown, with v being the top vantage point of the VP-tree. For example, v_{-+} in the top right of Figure 4.1c refers to the vantage point splitting the chunk $(-, +)$ into chunks $(-, +, +)$ and $(-, +, -)$. The point v_{-+} itself is considered to be part of chunk $(-, +, +)$. Counting the vantage point towards the positive half is not always technically correct because it could land in the negative half of a subtree, disconnected from the rest of the chunk. For instance, the two points v and v_+ in Figure 4.1c are counted towards chunk $(+, +, +)$ even though they lie outside the red area containing the rest of the points. But this minor inaccuracy should not matter too much in practice. What matters is that most chunk points have at least *some* close neighbors within the chunk. The spatial partitioning of the VP-tree should provide a sense of locality, even if chunks have multiple disconnected parts, like chunk $(-, -)$ in Figure 4.1b or chunk $(-, -, +)$ in Figure 4.1c.

A proximity graph can be built on the data in two steps. First, We find close neighbors among the points within each chunk and connect them. Afterward, the gaps between chunks need to be bridged by inserting connections between points from different chunks. Neighbors within each chunk can be found in a brute-force style, by computing the distances between all points in a chunk are computed exhaustively. The top-k neighbors of each point are then added to the proximity graph. Alternatively, a version of Relative NN-descent can be run on the set of chunk points to form in-chunk connections between them. Instead of building a single proximity graph, it might be beneficial to create a layered HNSW-like structure. To do so, a random level is sampled from $[-\ln(U[0, 1]) \cdot \text{level_norm}]$ for every point in the dataset, resulting in the same density at each layer as in an HNSW. On each level, a VP-tree can be built among the points of that layer, which is chunked and connected into a proximity graph. Every node on a higher level links to itself on the graph layer below to support a top-down greedy search.

4.1.1 Chunk Stitching

A VP-tree chunk is a set of points, but it can also be thought of as a volume enclosed by surface segments of d -dimensional hyperspheres. In the 2-dimensional case displayed in Figure 4.1 these surface segments are the black circular arcs enclosing the differently colored chunks. We call two chunks *sibling chunks* if they are at the same depth and differ only in the last element of their identifier (e.g., $(-, +, -)$ and $(-, +, +)$). One of two sibling chunks is called positive, and the other is called negative, depending on the last symbol in their identifier. Two sibling chunks are always separated by surface segments of a hypersphere that has a vantage point in the positive chunk as its center. We call these surface segments the *seam* between the two chunks.

Assume that we connect neighbors within each chunk to form a proximity graph. If we want to merge the proximity graphs of two sibling chunks into one, we need to connect nodes from both sides of the seam to each other. This process is called *chunk stitching*. After stitching a seam, the resulting merged graph represents a chunk having the identifier of its children without the last symbol. Note that there are 2^{depth} unique chunks at each depth. If we use a VP-tree to divide our data into $n_c = 2^{depth}$ chunks for some fixed depth, there are $\frac{n_c}{2}$ seam stitching operations, each merging two chunks. They result in $\frac{n_c}{2}$ chunks at $depth-1$. This process can be repeated until depth 0 is reached, where only a single proximity graph is left that encompasses the entire dataset. During this process, each seam dividing the data into different subtrees of the VP-tree has been handled exactly once. Consider the chunks in Figure 4.1c which can be merged in 3 waves. In the first wave, the following 4 merge operations happen: $(+, +, +) \cup (+, +, -) \rightarrow (+, +)$ and $(+, -, +) \cup (+, -, -) \rightarrow (+, -)$ and $(-, +, +) \cup (-, +, -) \rightarrow (-, +)$ and $(-, -, +) \cup (-, -, -) \rightarrow (-, -)$. In the next wave, only $(+, +) \cup (+, -) \rightarrow (+)$ and $(-, +) \cup (-, -) \rightarrow (-)$ need to be merged. Finally, $(+) \cup (-) \rightarrow ()$ are merged, creating a proximity graph that spans the entire dataset. Within each wave, merges can be executed in parallel.

The chunk stitching algorithm is initialized by creating a data structure that holds a list of neighbors for each point in the dataset. Each list can hold up to m_max neighbors. Next, we build a VP-tree on the entire dataset. A *max_chunk_size* parameter specifies the maximum chunk size of all initial chunks. These initial chunks are created by traversing the VP-tree and recursively splitting it into subtrees, as long as the chunk represented by the current subtree has more than *max_chunk_size* elements in it. Because the VP-tree is only used for space partitioning, the build process does not need to split subtrees with less than *max_chunk_size* elements any further. Once the initial chunks are formed, we exhaustively calculate the distances within each chunk and store the *same_chunk_m_max* closest neighbors of each point in the graph structure. The *same_chunk_m_max* parameter is chosen to be smaller or equal to m_max (the maximum number of neighbors). A smaller *same_chunk_m_max* parameter leaves room for cross-chunk connections that are not among the nearest neighbors but are essential for the graph's connectivity.

To stitch two chunks, our methods generate a list of stitching candidate pairs, consisting of a point p'_+ from the positive half and p'_- from the negative half. We insert bidirectional connections into the graph for each candidate pair. The insertion succeeds if there is still space in the respective neighbor list or if a neighbor with a distance greater than $d(p'_+, p'_-)$ can be replaced.

We present 4 methods for generating these stitching candidates. For the sake of simplicity, they are named *Method 1*, ..., *4*.

Method 1 randomly samples a fraction of points from the negative half. Starting from each of the sampled points p_- we greedy-search within the negative chunk towards the vantage point in the positive chunk p_+ . The search returns a point p'_- in the negative half. Next, we search from the vantage point of the positive half towards p'_- to obtain a point p'_+ . The points p'_- and p'_+ are on opposing sides of the seam and are hoped to be close to each other. They are added to the stitching candidates.

Method 2 is a variation of Method 1. Instead of searching toward the vantage point in the positive chunk, we pick a random p_+ within it. The negative half is searched from p_- to p_+ to obtain p'_- , then we search within the positive half from p_+ towards p'_- , resulting in p'_+ . The pair (p'_-, p'_+) is added to the stitching candidates. Method 2 could introduce more variety into the stitching candidates because the searches do not always target the same point. The idea behind method 1 is that the searches within the negative half would hit the surface of the positive chunk from many directions, like asteroids falling toward Earth's center of gravity. But, especially in higher dimensions, this analogy might not work, and random candidates like in method 2 could work better. Consider the seam between chunks $(+, +, +)$ and $(+, +, -)$ in Figure 4.1c. With Method 1, the top side of the seam is much more likely to receive stitches than the right side, while the additional randomness of Method 2 could increase local diversity.

Method 3 is similar to Method 2, but instead of sampling two points p_- and p_+ in every iteration, we sample a batch of X points on each side of the seam. Let us call these batches $(p_{1-}, p_{2-}, \dots, p_{X-})$ and $(p_{1+}, p_{2+}, \dots, p_{X+})$. All distances between point pairs $(p_{i-}, p_{j+}) \forall i, j \in 1, \dots, X$ from different sides of the seam are calculated. Then we select only the X pairs with the lowest distance and use them for a bidirectional greedy search connection attempt like in Method 2. Note that some sampled points on each side can participate in multiple searches, while others get discarded because they are not close to any sampled point on the opposing side of the seam. The multi-sampling could speed up the search by already starting with pairs of points that are on average closer to each other than in Method 2. This could also reduce the risk of running into a local optimum early on because the selection process can skip over dead ends within a chunk. However, selecting pairs with the lowest distance can also have disadvantages. If there are two chunks with a densely populated region on the seam between them, Method 3 is likely to favor point pairs from this region during sampling, decreasing the diversity of search entry points.

Method 4 is a variation of Method 2, but uses the greedy search from Algorithm 1 with $ef > 1$. This means that every search returns up to ef nearest neighbors instead of just a single one. In each iteration, we pick random points p_- and p_+ from the negative and positive chunk. We perform a greedy search with $ef > 1$ from p_- towards p_+ , resulting in up to ef candidates $p'_{1-}, p'_{2-}, \dots, p'_{ef-}$ on the negative side of the seam. We pick the one closest to p_+ and search towards it within the positive chunk, starting from p_+ with the same ef parameter. This produces candidates $p'_{1+}, p'_{2+}, \dots, p'_{ef+}$. We insert up to ef connections between the closest point pairs from both search results. Conducting the search with a greater ef parameter could help with local diversity and is less likely to get stuck in a dead end, but is also more expensive.

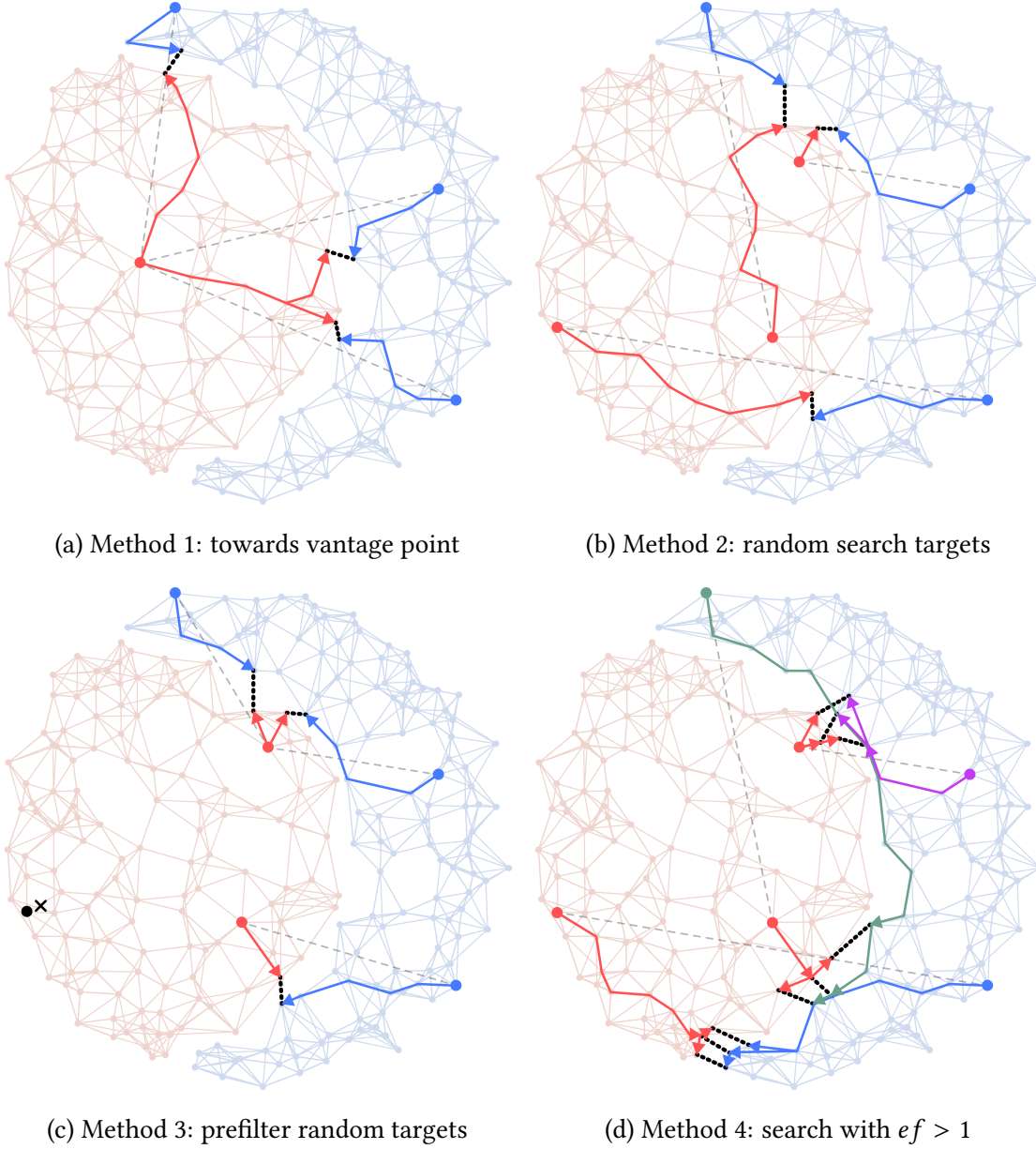


Figure 4.2: Visualization of the 4 chunk stitching methods generating stitching candidate pairs between a positive (red) and a negative chunk (blue).

Figure 4.2 visualizes the 4 proposed stitching methods. We generated a 2-dimensional dataset of 256 points and divided it into two chunks by constructing a VP-tree. These chunks represent a positive half (left side, red) and a negative half (right side, blue). Via exhaustive distance calculations, each point is connected to the 6 nearest neighbors within its chunk, forming two disconnected proximity graphs. The starting points of greedy searches within the positive and negative chunk are displayed as red and blue circles. Dashed lines are drawn between points where greedy searches start towards the opposing half. Arrowheads point at the stitching candidates found by the different methods. Thick, black, dotted lines are drawn between each pair of stitching candidates that are found.

For Method 1 in Figure 4.2a all searches involve the vantage point in the positive half. Using Method 2, searches are directed at random points in the positive chunk instead, but the search paths can be long sometimes, as seen in Figure 4.2b. Method 3 tries to fix this by sampling $x = 3$ points at once, as shown in Figure 4.2c, and only conducts searches for the 3 closest point pairs. One of the 3 randomly sampled points in the positive chunk is drawn in black. This point is not involved in any search because each starting point in the negative half is closer to a different starting point in the positive half. In Figure 4.2d the search paths of method 4 are colored differently in the negative half because they partly overlap. In this image, greedy searches with $ef = 3$ are shown. Each search returns 3 nearest neighbors that are used as stitching candidates. Note that the graphics for method 2 and method 4 use the same starting points, but the locations of the stitches differ quite a bit. This is because greedy search with $ef = 1$ as used in Method 2 is more likely to get stuck in a local optimum than Method 4 with $ef = 3$.

All stitching methods rely on the fact that we generate stitching candidate pairs first and only afterwards actually try to insert bidirectional connections into the graph. Duplicate candidate pairs do not lead to double insertions. This strict ordering ensures that none of the searches that yield stitching candidates cross the seam.

4.1.2 VP-Tree Ensemble

Stitching chunks is tricky. There are a lot of adjustable parameters, and it depends on the spatial structure and density of the data. In higher dimensions, it is not clear if finding points that are "close to the seam" makes sense. The curse of dimensionality tells us that almost all points in the positive half are going to be close to the seam in higher dimensions when a uniform distribution is assumed.

Therefore, we propose *VP-tree ensembles* as another method for building proximity graphs. It also relies on a vantage point tree dividing the dataset into chunks, but it does not perform any searches during the construction process. Instead of using chunks produced by a single VP-tree, it builds multiple VP-trees – an *ensemble*. Each tree is divided into chunks that partition the data space differently because vantage points are chosen randomly. For each chunk, we find connections within it, like described before. This causes every VP-tree to produce a different graph that has gaps between its chunks because no inter-chunk connections are ever made. Instead of stitching these seams, we can simply merge the graphs of multiple VP-trees together on a point-by-point basis. This creates a global graph where the neighborhoods of each point consist of the best in-chunk neighbors found in any of the VP-tree iterations.

Imagine any two points from the data that are close to each other. Even if there is a seam between them in some of the VP-trees, it is highly unlikely that they will end up in different chunks in *every* iteration. This is due to the random data partitioning of a VP-tree. Because the two imagined points are close to each other, they eventually make it into a shared chunk, and a bidirectional connection between them is attempted to be inserted into the merged global graph.

The *VP-tree ensemble* algorithm is initialized by creating an empty graph where every point is assigned an empty list of neighbors with a maximum capacity of m_{max} . At no point

in time, the constructed graph contains more than m_max neighbors per point. There are n_vp_trees iterations, and in each iteration a VP-tree is built on the entire dataset. Like in the stitching approach, we use a max_chunk_size parameter to only construct the VP-trees down to a certain depth. At this depth we convert the subtrees into chunks with a size between $max_chunk_size/2$ and max_chunk_size elements. Within each chunk, we determine the nearest $same_chunk_m_max$ neighbors per point and try to insert them into the corresponding neighbor lists of the global proximity graph. This list might already contain some of these neighbors from a previous iteration, so it has to be checked that a connection is not inserted twice. If the global neighbor list of a point already contains m_max neighbors, the insert operation only succeeds if it can replace a more distant neighbor.

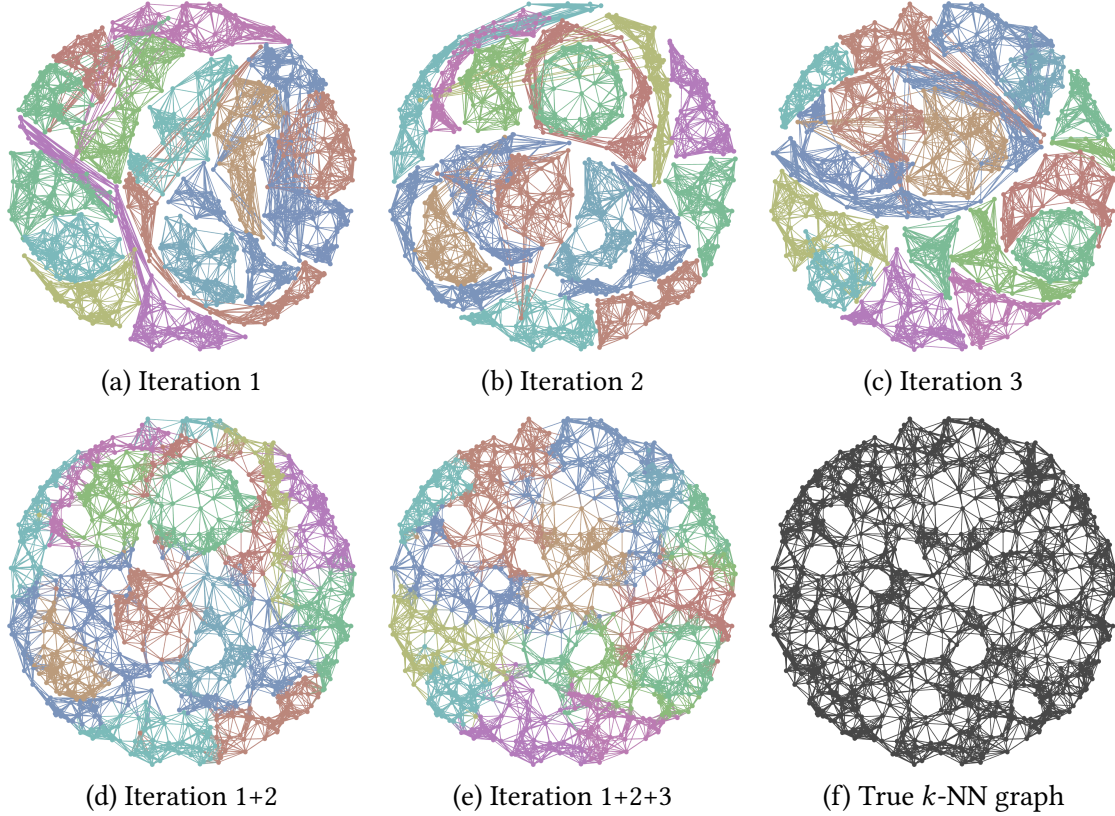


Figure 4.3: Merging graphs constructed by 3 VP-trees on a set of 1000 points. Parameters are $same_chunk_m_max = m_max = 12$ and $max_chunk_size = 100$, resulting in 16 chunks. In-chunk neighbors were found via exhaustive distance calculations. Figure 4.3d and 4.3e show the merged graphs with points colored according to their chunks in the VP-tree of the last iteration. The true k -NN graph ($k = 12$) is shown in Figure 4.3f for comparison.

Figure 4.3 visualizes how 3 VP-trees with randomly chosen vantage points result in 3 different chunk partitionings that can be merged into a single graph. In Figures 4.1a, 4.1b, and 4.1c, each point is connected to the 12 nearest neighbors within its chunk. In the displayed graphs, points within a chunk and their connections among each other share a color, such

that seams are clearly visible. Successively merging the neighbor lists from each iteration into a single proximity graph lets the gaps between chunks disappear, as shown in Figures 4.3d and 4.3e. The graph in Figure 4.3e is the result of merging the 3 graphs shown in the top row on a point-by-point basis. Already after 3 iterations, the graph structure seems to approach the true k-NN graph, which is displayed in Figure 4.3f.

Note that in this ensemble method, the computations for all chunks, as well as computations for all individual VP-trees, are independent of each other. This opens up two potential avenues for parallelization.

Because the brute-force method of calculating all distances within a chunk scales poorly with the chunk size, we developed a variant of the VP-tree ensemble method that applies the RNN-Descent algorithm on its chunks instead. In this case, the neighbor lists within each chunk are randomly initialized to *same_chunk_m_max* neighbors among the chunk points. With the two RNN-Descent iteration count parameters t_{outer} and t_{inner} , the build time and quality of in-chunk neighbors can be controlled. Because RNN-Descent uses the RNG-property [OM23], there could be the additional advantage of keeping some longer edges so that clustered data stays better connected.

4.2 Benchmark Data

To evaluate and compare the proposed methods, we use data from a subset of the *LAION-5B* dataset [Sch+22]. The *LAION-5B* dataset is a collection of 5.85 billion image-text pairs. A subset containing 2.32 billion pairs with text identified as English is also referred to as the *LAION-2B* dataset. On these image-text pairs, CLIP-embeddings can be trained. CLIP stands for Contrastive Language Image Pretraining [Rad+21]. The embeddings encode the shared semantic content of the image and text of each pair into a d -dimensional vector space. To achieve this mapping, two separate encoders are trained for image and text content. An image is fed through an image encoder, which is typically a convolutional neural network, as described by Lecun et al. [Lec+98]. The text is usually encoded through some kind of transformer architecture [Vas+23]. Doing so, one ends up with two vectors v_{text} and $v_{image} \in \mathbb{R}^d$. Because the image and the text of a pair should have some shared content, one aims to achieve a small distance between the two vectors. On the contrary, images and texts that do not fit together should ideally end up with very different encodings. This can be achieved by contrastive loss functions like the *InfoNCE Loss* [OLV18]. Calculating the contrastive loss for the encodings of an image-text pair allows us to nudge the model weights of the encoders into the desired directions.

Because semantic "meaning" is a complex thing, capturing it via embedding vectors requires a high number of dimensions, typically in the hundreds. The International Conference on Similarity Search and Applications (SISAP) hosts a yearly challenge about k -ANN search. The SISAP 2024 challenge committee [SIS24] provides 768-dimensional CLIP embeddings for a subset of 100 million points from the *LAION-2B* dataset. Each point is a normalized vector of 768 16-bit floating point numbers with scalar elements in the range $(-1, 1)$. In addition to this *100M* dataset, subsets of *10M* and *300k* points are officially provided as well. In this thesis, we use differently sized subsets of this data to evaluate search graph performance. To see the relative effect of model parameters on measured metrics,

we first use a *100K* subset of the provided data. Finally, we evaluate all approaches on the provided *10M* dataset.

The data is a suitable test bed for our graphs because it is high-dimensional, contains a large number of points, features real world data, and is normalized such that computing the cosine-similarity amounts to a simple dot-product calculation. The SISAP Challenge Committee [SIS24] also provides sets of 10000 queries (768-dimensional 16-bit floating point vectors) and their 1000 nearest neighbors, which can be used for evaluation. We use this 10K query set to assess the search time and recall of the graph structures built in our benchmarks.

For graph construction and search, we converted the data into 32-bit floating point numbers defined as *binary32* by the IEEE [IEE08]. The *10M* dataset provided by SISAP actually contains exactly 10,120,191 vectors and occupies a space of 31.09 GB in its uncompressed form. Note that in the context of this thesis we do *not* really care about the content of the CLIP-embedding data in particular, and just use it to test the proposed methods. While it would be interesting to see which image-text pairs are returned as the nearest neighbors of some query, this thesis only uses the distance of the vectors to each other and not their specific content.

4.3 Implementation

In the scope of this master thesis, reference implementations of a *VP-tree* [Yia93], an *HNSW-index* [MY18] and a *Relative NN-Descend graph* [OM23] have been developed in the Rust programming language [MK14].

Creating our own implementations allows us to keep the indices built by traditional means – e.g., HNSW construction via sequential insert – comparable to the same graph structure built by the new methods. For example, an HNSW and a multilayered VP-tree ensemble have exactly the same memory layout in the developed Rust library. Also, the functions for conducting search and distance calculations map to the same hardware instructions. This isolates the way an index is built (proposed methods vs. existing methods) as the main source of variation when conducting benchmarks.

The custom implementations also give us a fine-grained control over synchronization, allocation strategies and tunable parameters. Behind conditional compilation flags we are able to track the number of distance calculations during the build and search process, and emit events such that visualizations of the structures and their build processes can be produced.

Our reference implementation of the HNSW is benchmarked against 4 external implementations. *Hnswlib* [MY+18] and *Faiss* [Dou+19] are written in C++, while the *RustCv* [Wor21] and *JpBoth* [Bot24] are HNSW libraries developed in Rust as well. The *Hnswlib* C++ library and the corresponding Python package are relatively popular and were developed by Malkov and Yashunin [MY18], the original inventors of the HNSW idea.

4.3.1 Construction Parameters

Our own HNSW implementation follows the paper by Malkov and Yashunin [MY18] and has the following tunable build parameters:

- *level_norm* (named m_L in Malkov and Yashunin [MY18]) affects the height of the multilayered graph. During the construction process, the level for each inserted point is randomly sampled as $level = \lfloor -\ln(U[0, 1]) \cdot level_norm \rfloor$.
- *m_max* is the maximum number of neighbors on each layer. A special parameter *m_max_0* is used for the maximum number of neighbors on the bottom layer. Unless further specified, we stick to the recommendation of Malkov and Yashunin [MY18] keeping $m_max_0 = 2 \cdot m_max$.
- *ef_construction* describes the maximum size of the candidates list used in greedy-search when looking for neighbors to form connections within each layer.

The *level_norm* parameter is only explicitly settable in our own HNSW; the other libraries set it automatically to $\frac{1}{\ln(m_max)}$, a heuristic proposed by Malkov and Yashunin [MY18]. All external HNSW implementations support setting the *ef_construction* and *m_max* parameters. In the *RustCv* implementation, all parameters except *ef_construction* need to be specified at compile time. Furthermore, the *JpBoth* and *RustCv* implementations require the distance function to be set at compile time as well. This makes them slightly less flexible than our HNSW implementation. There are existing Python modules for *Hnswlib* and *Faiss*. For *JpBoth*, *RustCv*, and our own HNSW, a Python wrapper was created to test them under the same conditions.

The VP-tree construction algorithm subdivides the data recursively into subtrees. Sometimes it is enough to perform splits only up to a certain depth and treat all points in subtrees at this depth as unordered batches of points. This is particularly useful when using the VP-tree to "cluster" the data into distinct chunks. We provide a *max_chunk_size* parameter in the VP-tree implementation to only split a subtree into further subtrees if the chunk has more than *max_chunk_size* elements in it. We also expose a *n_candidates* parameter, which lets the VP-tree pick the best vantage point among a certain number of candidates at each split. If this parameter is set to 1, a single random point is chosen as the vantage point.

Our *Relative NN-Descent* implementation uses the same parameters as outlined in Algorithm 2. Namely, they are:

- *t_outer* refers to the number of outer loops.
- *t_inner* refers to the number of inner loops.
- *m_initial* is the number of initial random neighbors assigned to each node at the start of the algorithm.
- *m_pruned* is the maximum in- and out-degree of each node after adding reverse edges and pruning the number of connections at the end of every outer loop (except for the last one).

The Rust implementations for the *chunk stitching* and *VP-tree ensemble* approaches described in this master thesis share the following configurable parameters:

- *max_chunk_size* controls the size of the chunks the vantage point tree is split into. In reality chunk sizes are going to be between $\text{max_chunk_size}/2$ and max_chunk_size .
- *m_max* is the maximum number of neighbors each node can have in the resulting graph. If a multilayered graph should be built, a *m_max_0* parameter can be configured separately for the lowest layer, just like for the HNSW implementation.
- *same_chunk_m_max* describes the number of neighbors a point is connected to within its own chunk. It is typically much lower than *m_max* to allow for some headroom in the neighbor lists that is filled by stitching or multiple iterations of the ensemble method.
- *n_candidates* refers to the number of vantage point candidates considered during VP-tree construction.

In addition to these, the stitching method allows the user to select one of the 4 methods, which vary in the way stitching candidates are found. All the stitching methods use a *fraction* parameter that describes what fraction of points in the negative and positive chunks is involved as starting points for the searches towards the seam. For stitching method 3, the *X* parameter specifies the number of candidates on each side of the seam to be considered in each batch, allowing us to select the best searches among pairs of them. For method 4 the *ef* parameter specifies the number of points returned from each greedy search.

Our Rust implementation of the VP-tree ensemble uses the *n_vp_trees* parameter for the number of trees to be built. It also lets you choose a *level_norm* parameter to create multilayered graphs if set to values greater than zero. It works in the same way as for the HNSW. The method for finding in-chunk neighbors can also be selected. You can either perform brute-force distance calculations between all points of the chunk and select the *same_chunk_m_max* best neighbors or use RNN-Descent instead. If RNN-Descent is chosen, the number of iterations *t_inner* and *t_outer* can be freely configured. In this case, *m_initial* and *m_pruned* are set to the *same_chunk_m_max* parameter.

All presented methods use some form of randomness. We use a pseudo-random number generator based on the *ChaCha*-cipher [Ber08] that is seeded by an integer to create reproducible results (in a single-threaded context). In addition to the parameters listed above, our models allow you to specify any distance function $\text{dist} : (\mathbb{R}^d, \mathbb{R}^d) \rightarrow \mathbb{R}$. By default, we use the simplified cosine distance $\text{dist}(a, b) = 1.0 - a \cdot b$ based on the dot product for the spherical *LAION-2B* data.

4.3.2 Rust Implementation Details

This section talks about a few engineering decisions that have been made while implementing the methods of this master thesis in the Rust programming language. There are a few guiding principles that we followed to increase performance. Firstly, data should not be duplicated. Especially when dealing with high dimensional data and large dataset sizes, the data for all points should be stored in a single place. When a dataset is loaded, we never want to store the literal data points in a structure like an HNSW or a VP-tree. Instead, it is better to access each point in the dataset through a unique 32-bit ID. Using 32-bit IDs is

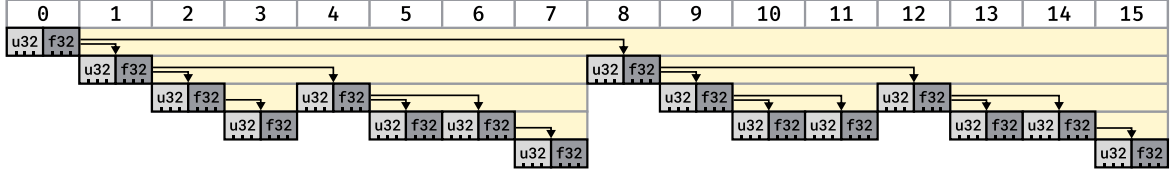


Figure 4.4: Flat memory layout of a VP-tree on a dataset of 16 points. The top row displays the indices $0, \dots, 15$ of the nodes in the slice. Each node at an index i is the parent of a region of n_{dsc} nodes marked in yellow to the right of it. Within this region it has up to two child nodes. A left child node is always located at index $i + 1$, while the right child node is at index $i + \lceil \frac{n_{dsc}}{2} \rceil$. Since every node occupies 8 bytes, this VP-tree can be built using just 128 bytes of memory

sufficient for all practical datasets with $n \leq 2^{32} \approx 4.9$ billion elements. Only IDs are stored in the index structures and the neighbor lists within.

Memory locality is another principle that is important for high-performance data structures. In Rust, memory of a dynamic size is usually allocated from a global heap allocator. It should be preferred to allocate large blocks of contiguous memory in favor of individual small allocations that could be spread out all over the heap. Operating on a dense local slice of memory decreases the chance of cache misses. Therefore, we aim to allocate large chunks of contiguous memory upfront wherever possible and shape it into the desired structure. Our VP-tree for a dataset of n elements is represented as a slice of Node structs with a memory layout struct `{id: u32, t: f32 }`. Here, the `id: u32` field refers to the 32-bit ID of the point in the dataset. The `t: f32` field refers to the threshold distance that separates the subtrees descending from this node. But where is the tree structure? Don't we need pointers to child nodes? The entire structure of the tree is implicit. Figure 4.4 shows how the nodes are laid out in memory. The index of a node in the slice of memory uniquely determines its place in the vantage point tree. In the figure, arrows point from parent nodes to their left and right child nodes. Indices of child nodes do not need to be stored. They can be computed on the fly using the index of the parent node and a number n_{dsc} that describes how many descendants the parent node governs. The tree can be traversed starting at the root node which always has index $i = 0$ and $n_{dsc} = n - 1$. A similar structure is proposed by Yianilos [Yia93].

In our VP-tree a left and right subtree never have the same size. The right subtree always contains 1 or 2 nodes more. This decision makes it easier to construct same-sized chunks from the slice of nodes. In Figure 4.4 for example, the tree can be easily split into 4 same-sized chunks when splitting at indices 4, 8, and 12, with each chunk covering a subtree, and for left subtrees, one or more upper vantage points.

The construction of the VP-tree can happen completely in-place using the slice of memory. At every step, a vantage point is chosen, and its node is swapped with the first element of the slice. Next, distances to each other node are computed and stored in the respective `t: f32` field. The nodes are arranged into a left and right subtree using the *Quickselect* algorithm [Hoa61], and the smallest distance among points in the right subtree is stored in the `t: f32` field of the vantage point at index 0 of this slice. If necessary, other metadata can be stored in the nodes as well. For example, when constructing a VP-tree for the nodes of an HNSW layer, we store the layer index in the VP-tree nodes as well.

In an HNSW, each layer consists of a proximity graph. Such proximity graphs can be represented by a *list-of-lists* approach. This means that there is a list of nodes, and each node stores a list of neighbors. There are multiple layers so the entire HNSW can be thought of as a *list-of-lists-of-lists*.

When constructing an HNSW, we frequently replace the most distant neighbor in a neighbors list with a better one (cf. Algorithm 1). This can be done efficiently in $O(\log(m_{\max}))$ time if we organize the neighbors list as a Max-Heap. We store in these heaps the distance to each neighbor and its index in the current HNSW layer. When constructing an HNSW on a dataset of n points, we first determine the insertion level for each point. This tells us the number of graph nodes (n_{layer}) in each layer upfront. Every node in an HNSW layer can have a maximum of $m_{\max}$ or $m_{\max_0}$ neighbors. Using this knowledge, we allocate a single slice of memory for each layer holding all neighbor lists. This slice has enough space for $n_{\text{layer}} \cdot m_{\max}$ index-distance pairs in total. The index of each point in a layer then determines the index range in this big slice of memory that should be treated as its neighbors list. The same allocation scheme is used when building a multilayered graph by chunk stitching or a VP-tree ensemble.

For RNN-Descent graphs, it is not so easy to put an upper bound on the number of neighbors. Therefore, our reference implementation sticks with a simple list-of-lists approach where each list is individually allocated.

Parallelization

To construct an HNSW with multiple threads, *Read-Write-Locks* are used to synchronize access to the neighbor lists. Because locking and unlocking them introduces some overhead during the search, we provide a function that converts the parallelized HNSW into a representation without locks, once it is built. For the parallelization of the Relative NN-Descent algorithm, locks are necessary as well. However, here we take a different approach and store a list of `Mutex<>` adjacent to the data structure, such that it can be thrown away after the build process.

We parallelize the construction of a VP-tree in two different ways. When a vantage point is picked, all the distances to it can be computed in parallel on multiple threads. Afterwards, the two resulting subtrees can be computed independently as well. The VP-tree ensemble method does not require any locks or other synchronization structures, because of the strict separation between chunks. Because there is typically a lot of work to do for the chunks of a single VP-tree on a large dataset, we parallelize across the chunks but perform the $n_{\text{vp_trees}}$ iterations sequentially.

Our multithreaded VP-tree ensemble and HNSW implementations originally used thread pools to split the work equally among a fixed number of threads. However, we found that using a work-stealing approach using the *rayon* crate by Matsakis [Mat15] worked better since it can dynamically distribute work among threads depending on how fast they proceed.

5 Results

Performance benchmarks for all presented methods have been conducted on a machine provided by the computer science department of TU Dortmund University. The machine features an Intel Xeon E5-2690 v4 CPU that supports 56 threads on 28 cores. Each core has its own L1 and L2 cache, with 896KB and 7MB of memory, respectively. There are two instances of a shared 70MB L3 cache and a total RAM of 540GB. In each of our benchmarks, search indices are built on subsets of the *LAION-2B* dataset [Sch+22; SIS24]. Following the index construction, we perform approximate nearest neighbor searches on the constructed graphs using a set of 10,000 query points. This 10K query set also consists of randomly sampled 768-dimensional vectors from the *LAION-2B* dataset but does share points with the datasets used to build the search indices.

We track three key performance metrics: index *build time*, *search time*, and *recall*. In general, we search for the $k = 30$ nearest neighbors of each point in the 10K query set and calculate the average recall and search time per query. Four graph construction methods are compared: HNSW-indices, RNN-Descent, Chunk stitching, and the VP-tree ensemble approach. The cosine distance, computed via the dot product, is used as a distance measure for graph construction and searches. To observe differences in the measured metrics, the build and search parameters of all models are varied. Since all our methods retrieve the approximate nearest neighbors via greedy search, the *ef* search parameter can be almost always increased to gain recall at the cost of search time.

This chapter is split into two parts. First, we look at each class of models individually and vary parameters to get an idea of how they affect the captured metrics. Here, we use our own Rust implementations of all methods and only a 100K subset of the data ($n = 100,000$) to build the search indices. For each set of parameters, indices were built 10 times, and the metrics were averaged across all processes. Still, there might be random fluctuations, especially if a parameter itself does not contribute much to the total variance of the measured metrics. If not further specified, greedy searches with $k = 30$ and $ef = 60$ on the 10K query set are used to report recall and search times. The experiments on the 100K dataset establish a feeling for reasonable parameter ranges and expectable metrics.

In the second part of this chapter, selected parameterizations of all model classes are constructed on the 10M dataset ($n = 10,120,191$) provided by the SISAP Challenge Committee [SIS24]. We also benchmark our HNSW implementation against the four external libraries to evaluate its competitiveness. For each graph that is built on the 10M dataset, searches for a whole range of *ef* parameters are run for the 10K query set, to find out if, and when a model is able to cross a recall threshold of 80%. Finally, we use all available cores of the machine to assess, how well the individual models can be constructed in parallel on multiple threads.

5.1 HNSW

We individually varied the construction parameters m_max , $ef_construction$, and $level_norm$ of our HNSW implementation to see what influence they have on the measured performance metrics *build time*, *search time* and *recall* while keeping all other parameters constant. The parameter m_max describes the maximum number of neighbors each node in an HNSW layer can have. For the bottom layer, twice as many neighbors are allowed ($m_max_0 = 2 \cdot m_max$). During construction, the best m_max neighbors are picked from a list of up to $ef_construction$ candidates returned from a greedy search.

As Figure 5.1 shows, the recall of our HNSW can be increased by increasing the m_max parameter. Recall values of more than 98% were achieved on the 100K dataset for $m_max = 40$. But storing a greater number of neighbors in the HNSW also leads to higher build times and search times. Figure 5.1 shows that there is an almost linear relationship between the m_max parameter and both time metrics. Depending on the application, there likely is a threshold when it does not make sense to increase the m_max parameter further because the payoff in terms of recall is diminishing while the cost keeps increasing linearly.

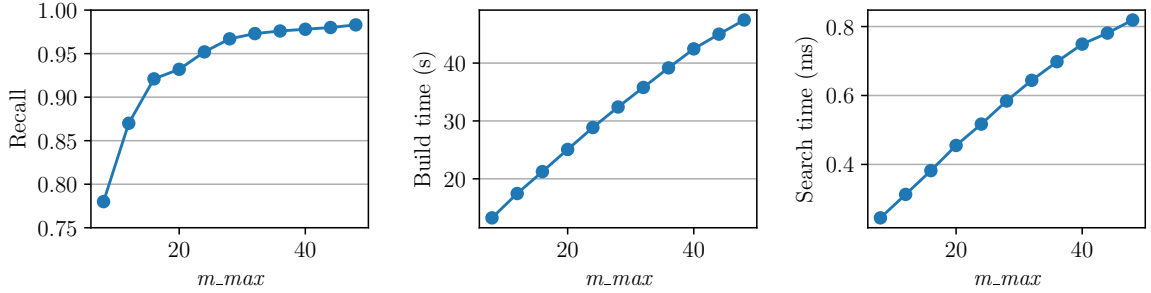


Figure 5.1: Effect of the m_max parameter in an HNSW with $level_norm = 0.3$ and $ef_construction = 40$ built on the 100K dataset.

As shown in Figure 5.2, there seems to be a small recall gain when increasing the $ef_construction$ parameter, with all other parameters kept equal. The recall gain starts to plateau when reaching a value of about m_max_0 . In contrast to increasing the m_max parameter, a greater $ef_construction$ value seems to not lead to higher search times, as the almost flat line in the third graph of Figure 5.2 indicates. This makes sense, considering that the final graph structure will never have more than m_max neighbors per node, even if a very large $ef_construction$ is chosen to consider more candidates during graph construction. The HNSW's build time seems to increase linearly with the number of considered candidates.

The $level_norm$ parameter controls how "tall" the layered structure of the HNSW becomes. As shown in Figure 5.3 there is no conceivable relationship between this parameter and the achieved recall. When $level_norm$ is chosen too high or too low, build and search times suffer. In our experiments, we found that a $level_norm$ of around 0.3 ± 0.1 leads to the best search and build times. Malkov and Yashunin [MY18] found similar values to be optimal and propose to set it to $\frac{1}{\ln(m_max)}$. In the experiment of Figure 5.3, this would be $\frac{1}{\ln(20)} \approx 0.33$. Note that the change in both search and build times is quite small, especially if you consider the difference between $level_norm = 0$ and $level_norm = 0.3$. This comes

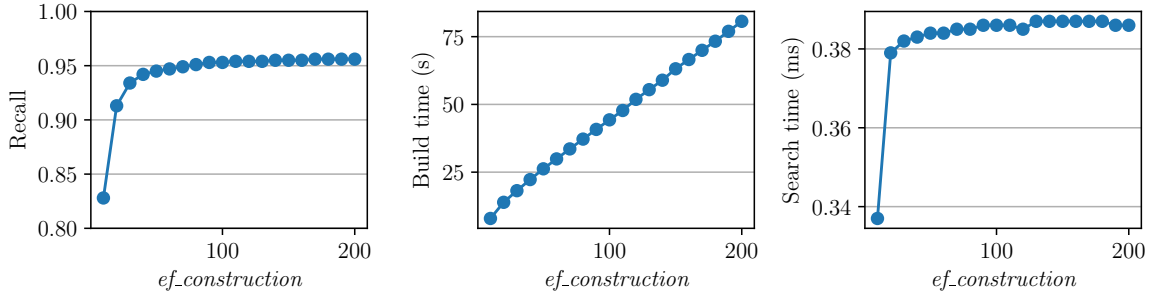


Figure 5.2: Effect of the $ef_construction$ parameter in an HNSW with $level_norm = 0.3$ and $m_max = 20$ built on the 100K dataset.

as a surprise to us, because we would have expected that the layered nature of the graph structure had a much bigger effect, especially on the search times. As visible in Figure 5.3, there seems to be a search time difference of less than 10% between a flat graph and a layered HNSW, in our experiments.

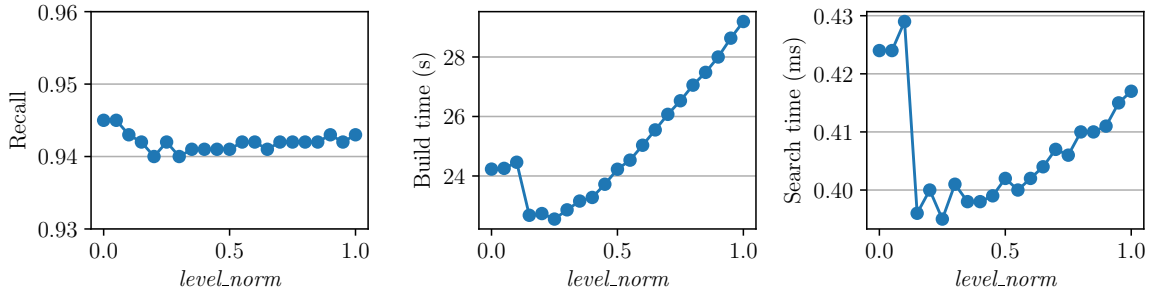


Figure 5.3: Effect of the $level_norm$ parameter between 0 and 1 in an HNSW built with $m_max = 20$ and $ef_construction = 40$ on the 100K dataset.

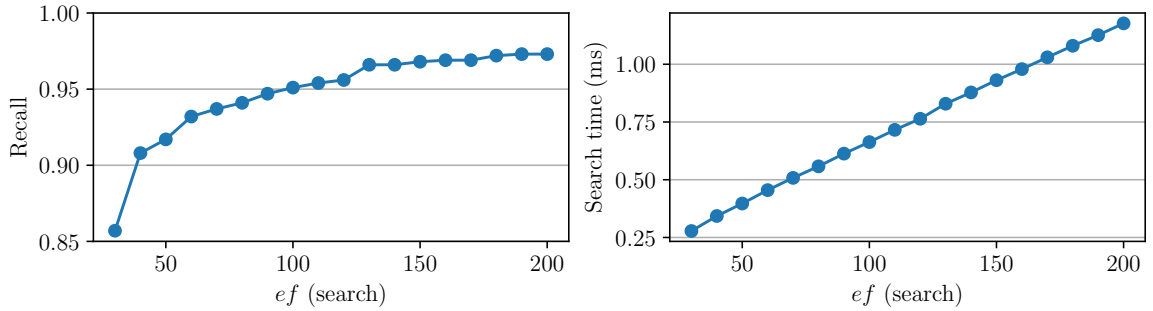


Figure 5.4: Recall and per-query search time for ANN-search with $k = 30$ and varying ef on HNSWs with $m_max = 20$, $ef_construction = 40$ and $level_norm = 0.3$.

A HNSW is searched via top-down greedy search, where the user specifies an ef parameter, indicating the number of points returned by the search. When k neighbors are wanted, it is possible to set $ef > k$ and select only the k closest neighbors from the ef returned points.

Figure 5.4 shows that for NN-searches with $k = 30$, increasing ef leads to a substantial improvement of the recall. Just like we saw for the $ef_construction$ parameter, there is an almost perfect linear relationship between the searches' ef and the search time. On the query set of 10,000 points, setting $ef = 30$ achieved only 85.7% recall, while for $ef = 200$, 97.3% of the returned neighbors were part of the 30 true nearest neighbors.

The size of the dataset also affects the metrics of an HNSW. For larger datasets, the build parameters m_max and $ef_construction$, or the search parameter ef need to be increased if the same recall should be achieved. We built HNSW indices for subsets of the *LAION-2B* dataset with sizes between 10K and 10M points. In Figure 5.5 it is shown, that the recall decreases from 0.97 for 10K points to only 0.85 for 10M points when the build parameters are kept constant.

The search times and per-point build times increase logarithmically with the size of the dataset, as indicated by the plots in Figure 5.5. Note the logarithmic scaling of the x-axis. This means that the total build time of an HNSW increases more rapidly than the size of the dataset increases, so for larger HNSWs, each individual insert operation is more expensive.

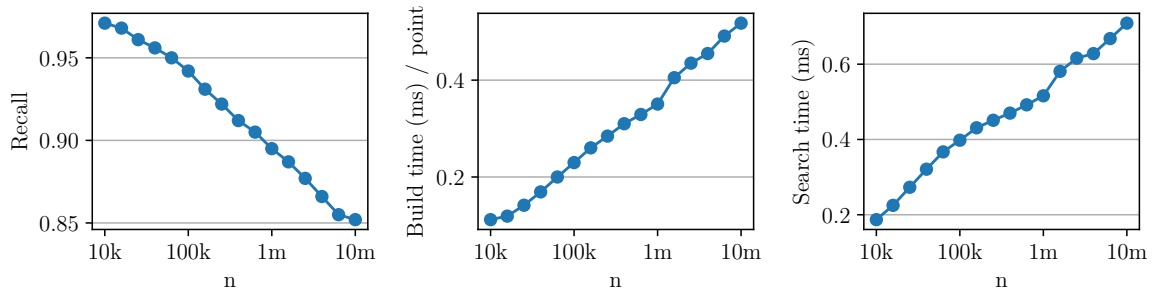


Figure 5.5: Effect of the dataset size n on the metrics achieved by an HNSW with $m_max = 20$, $ef_construction = 40$ and $level_norm = 0.3$. Note that the x-axis showing n between 10,000 and 10,000,000 is logarithmic, and the build time is reported as an average per point. This is the total build time divided by n .

5.2 Relative NN-Descent

Our implementation of the Relative NN-Descent algorithm has a few parameters that can be varied to strike a balance between build time, search time, and recall. First, we want to find out how many iterations are necessary for a graph built by RNN-Descent to converge to a good search index. The Relative NN-Descent algorithm consists of two nested loops, as shown in Algorithm 2. The outer loop is executed for t_outer iterations, and the inner loop t_inner times within each outer iteration. First, we look at graphs built with $t_outer = 1$, skipping the pruning step in the outer loop entirely. This allows us to observe only the effect of the descent step in the inner loop, updating the neighbors. Figure 5.6 shows the effect that an increasing number of descent steps has on the metrics.

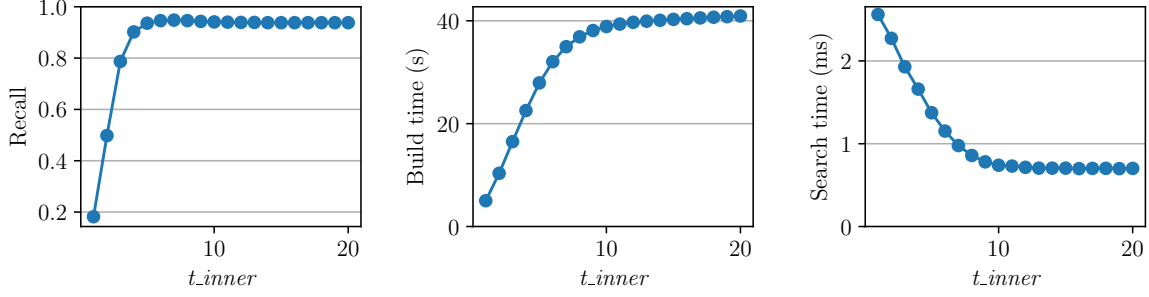


Figure 5.6: Effect of the number of inner loops t_{inner} when constructing a Relative NN-Descent graph with parameters $t_{outer} = 1$ and $m_{initial} = m_{pruned} = 40$. Note that there is no pruning applied, because there is only a single outer loop.

On the 100K dataset, the recall seems to reach a plateau at around 94% after 6 iterations already, as shown in the right graph of Figure 5.6. Interestingly, the nearest neighbor descent steps become cheaper over time as well, because the *is_new* flag in the algorithm prevents distance computations in neighborhoods that don't change much anymore. Even though the recall already peaks after 6 iterations, the rightmost plot in Figure 5.6 shows decreasing search times up to iteration 10, suggesting that the graph is still converging to a better solution. In the Relative NN-Descent algorithm, as outlined in Algorithm 2, pruning neighbors, and adding reverse edges back in, takes place in the outer loop in all but the last iteration. For this to take effect, $t_{outer} > 1$ has to be used.

Table 5.1: Relative NN-Descent: effect of t_{outer} for a graph with $m_{initial} = m_{pruned} = 40$ on the 100K dataset. NDC stands for the number of distance calculations.

t_{inner}	t_{outer}	build time	build NDC	recall	search time	search NDC
12	1	39.57s	213.09 M	0.941	0.70ms	1.88 K
6	2	39.35s	209.81 M	0.959	0.49ms	1.29 K
4	3	35.07s	176.26 M	0.964	0.53ms	1.36 K
3	4	33.41s	160.47 M	0.965	0.54ms	1.41 K
2	6	30.66s	136.86 M	0.964	0.54ms	1.41 K
1	12	27.89s	107.32 M	0.945	0.49ms	1.27 K

As shown in Table 5.1, we changed the number of outer loops while keeping the total number of inner loop iterations, $t_{inner} \cdot t_{outer}$ constant. Increasing the number of outer loops decreased the build time and number of distance calculations (NDC) performed during the build process. Even though executing the outer loop pruning step of the algorithm is additional work, it keeps the number of neighbors in check, which makes the descent step in the inner loops cheaper. Recall and search times are also slightly improved by having the pruning step, but we cannot see a strong effect past $t_{outer} \geq 2$.

5 Results

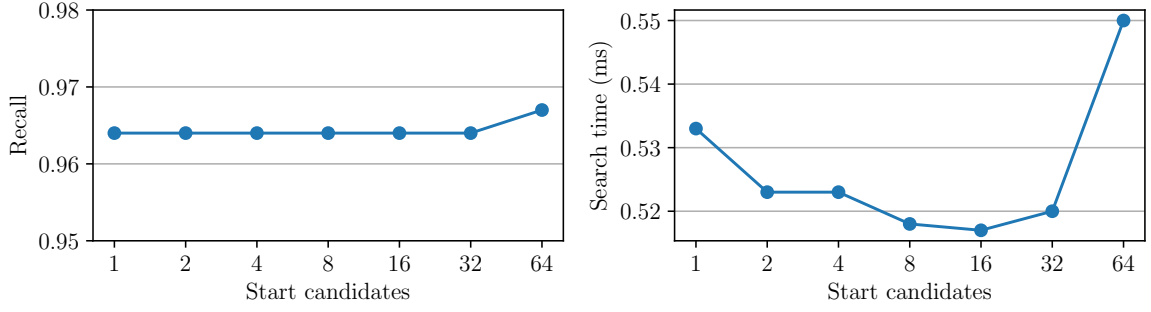


Figure 5.7: Greedy search with multiple start candidates on RNN-Descent graphs built on the 100K dataset with $m_{\text{initial}} = m_{\text{pruned}} = 40$, $t_{\text{outer}} = 3$, and $t_{\text{inner}} = 4$.

Single-layer proximity graphs like the ones produced by the Relative NN-Descent algorithm can be searched using multiple randomly chosen entry points. Figure 5.7 shows that there is virtually no effect on recall by doing so. However, using multiple entry points for the greedy search was found to improve search times slightly, as seen in the right plot of Figure 5.7. Using about 16 start candidates was optimal. Multiple candidates have a higher chance of starting the search right away from a point that is close to the query. But if too many candidates are chosen, the search becomes more expensive again.

When increasing the size of the dataset while keeping the parameters of the RNN-Descent algorithm constant, we observe that the recall decreases, while the per-point build times and search times increase. These relationships are shown in Figure 5.8. However, the recall drop seen with increased n seems to be lower than the decrease in recall we saw for the HNSW in Figure 5.5.

5.3 Chunk Stitching

We now take a look at the VP-tree chunk stitching methods proposed in this thesis. They all consider a randomly sampled *fraction* of points in the negative chunk as entry points for searches towards the positive half to form connections between the two.

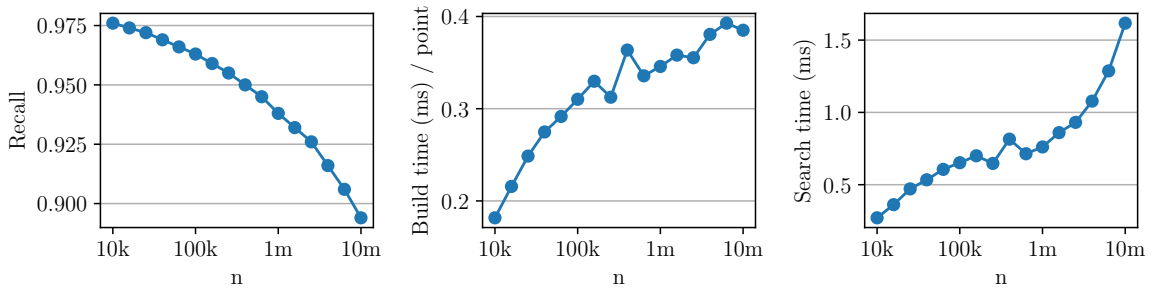


Figure 5.8: Effect of the dataset size n on metrics of a graph built by Relative NN-Descent with parameters $t_{\text{outer}} = 3$, $t_{\text{inner}} = 3$, and $m_{\text{initial}} = m_{\text{pruned}} = 40$ on the 100K dataset.

Method 1 searches from a random point in the negative half to the vantage point of the positive half and back. Method 2 searches from a random point in the negative half to a random point in the positive half and back. Method 3 considers batches of X random points from both chunks and conducts the X shortest searches between them. Method 4 is like method 2 but uses greedy searches with $ef > 1$, to generate multiple stitching candidate pairs at once.

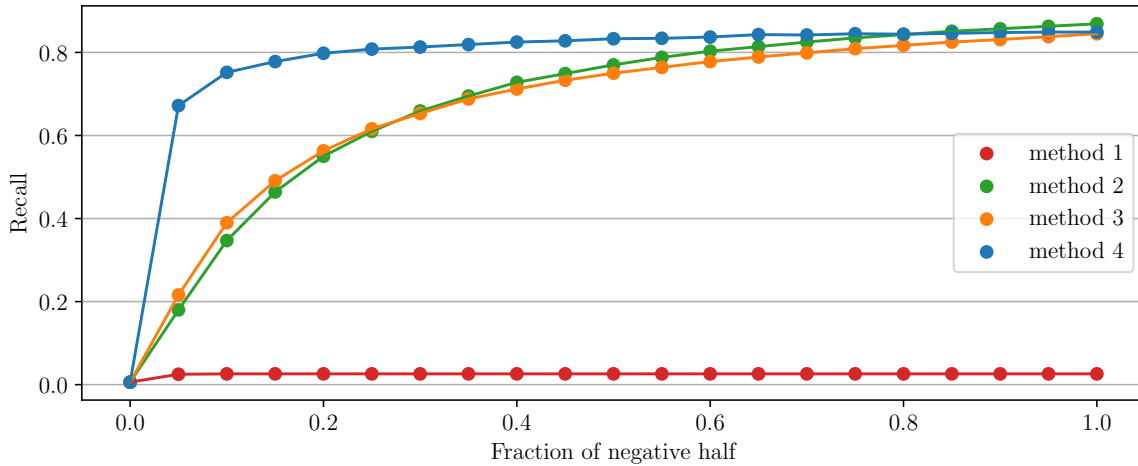


Figure 5.9: Recall of chunk stitching methods with varying *fraction* parameter, building graphs on the 100K dataset. Shared parameters are *max_chunk_size* = 256, *same_chunk_m_max* = 40, and *m_max* = 40. Method 3 used batches of $X = 10$ candidates in the positive and negative half to preselect the searches that should be taken. Method 4 uses $ef = 10$ to propose multiple stitching candidates at chunk borders.

Figure 5.9 shows how the recall increases for all stitching methods when a greater fraction of points from the negative half is sampled. With a maximum recall of only 2.60%, method 1 performs worst. Searching exclusively towards the center of the positive half does not seem to generate good inter-chunk connections for this high-dimensional dataset.

All other methods are able to achieve more than 80% recall. Unsurprisingly Method 2 and 3 show a similar recall curve in Figure 5.9. For lower *fraction* values, method 3 seems to be slightly better, while the simpler method 2 surpasses it when more points are visited. Method 4, which returns 10 candidates from each of its greedy searches, reaches higher recall values when looking at a smaller *fraction*.

At a sampled fraction of 20% it already surpasses a recall of 80%. However, if we look at the build times of these graphs in Figure 5.10, we can also see that method 4 is the most expensive one. Building a graph with method 4 and *fraction* = 0.2 is just as expensive as using method 3 while sampling 90% of points from the negative half. A greedy search that returns multiple points simply takes longer. In this comparison, methods 2 and 3 would achieve a better recall than method 4 for the same build time. Method 3 achieves better build times than method 2. Picking the best searches among X candidates on each side of the seam eliminates some longer-ranging searches, boosting the build time of method 3.

5 Results

It is unsurprising, that the graph construction times in Figure 5.10 scale linearly with the fraction of visited points, for all methods.

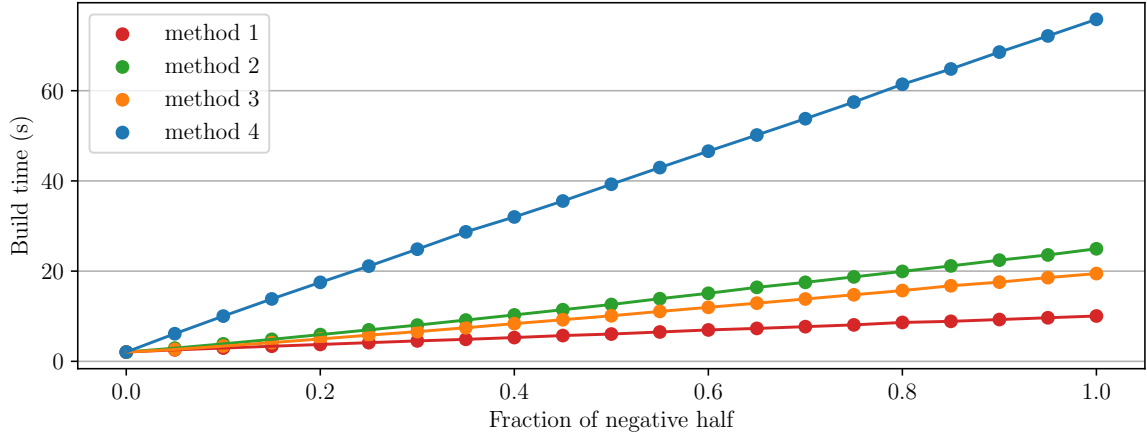


Figure 5.10: Build time of chunk stitching methods with varying *fraction*. For a detailed parameter description, see the caption of Figure 5.9.

Figure 5.11 shows the per-query search times of the graphs produces by the 4 different methods. We can ignore the results for method 1, even though it has the fastest search and build times. Because of its poor recall we consider the idea behind method 1 a failure and do not discuss it any further in this thesis. Method 4 beats the other two methods in terms of search speed by more than 20%. Its search time declines with an increasing *fraction*, while method 2 and 3 show the opposite behavior.

Figure 5.12 shows the effect of the chosen maximum chunk size for stitching methods 2, 3, and 4. To remove the big cost differences between the methods, we used a *fraction* parameter of 0.6 for method 2 and 3, while method 4 is assigned a lower *fraction* = 0.2.

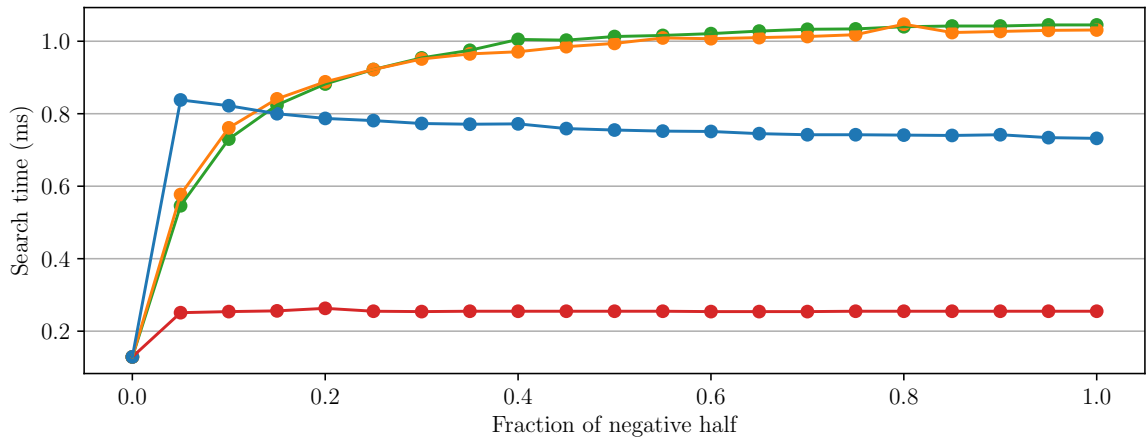


Figure 5.11: Search times of chunk stitching methods with varying *fraction*. For a detailed parameter description, see the caption of Figure 5.9.

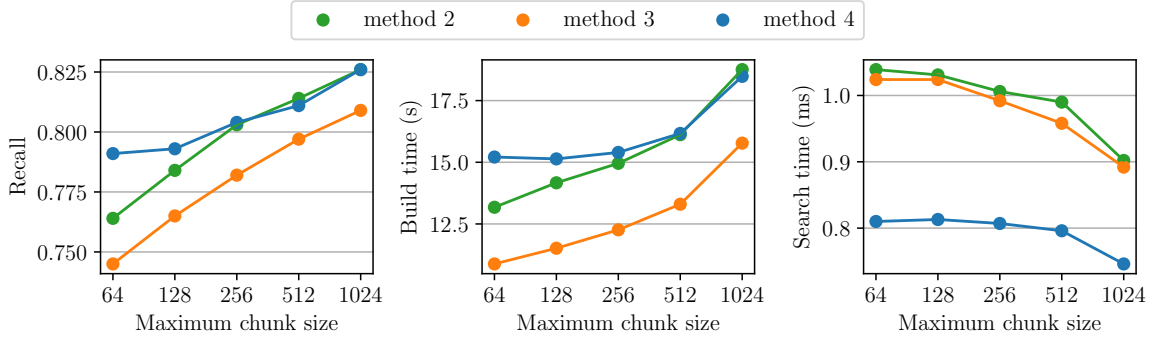


Figure 5.12: Chunk Stitching on the 100K dataset: effect of *max_chunk_size* on recall, search time and build time. The number of neighbors is set to *m_max* = *same_chunk_m_max* = 40. For method 2 and 3 *fraction* = 0.6 was chosen and for method 4 *fraction* = 0.2. Method 4 uses *ef* = 8 and method 3 sets *X* = 8.

For all stitching methods, recall and build time increase with larger chunk sizes, while the search time decreases. Returning multiple neighbors as stitching candidates in method 4 seems to be beneficial compared to the simple method 2; For a maximum chunk size of 512 points, recall and graph construction time were almost equal between the two, but method 4 features better search times.

Taking a look at Figure 5.13, we see that the build and search times of the stitching methods increase linearly with the number of neighbors. The increase in recall however, evens out past about 30 neighbors being stored per point in the graph. For the chosen parameters, the recall plateaus at around 75% for all 3 methods. This could indicate that there are some fundamental problems with the construction algorithm and that stitching is not really capable of doing high precision *k*-ANN search.

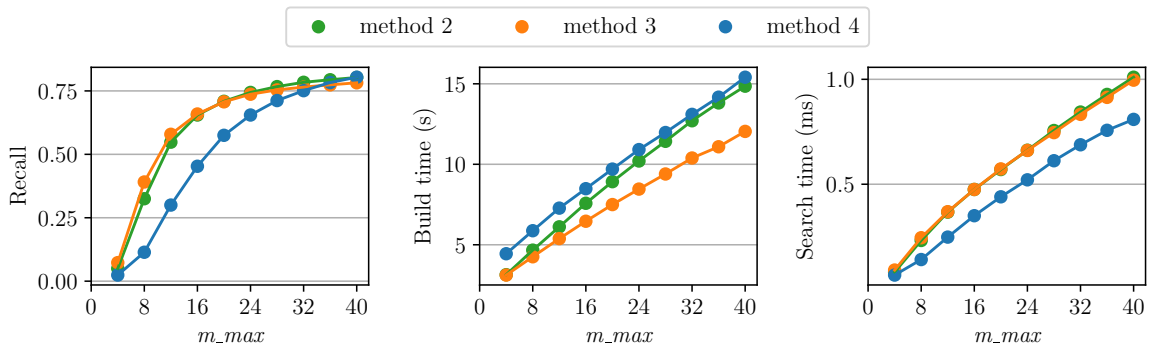


Figure 5.13: Chunk Stitching on the 100K dataset: effect of *m_max* on recall, search time and build time. Shared parameters are *same_chunk_m_max* = 40 and *max_chunk_size* = 256. For method 2 and 3 *fraction* = 0.6 was chosen and for method 4 *fraction* = 0.2. Method 3 uses *x* = 8 and method 4 uses *ef* = 8.

Our results suggest that it is possible to increase the recall by choosing higher *m_max*, *max_chunk_size*, and search *ef*, but this also increases the graph construction times signif-

icantly and is likely not competitive, compared to the metrics of an HNSW.

Regarding the *same_chunk_m_max* parameter, we found that setting it to values less than *m_max* did not have a positive effect. Doing so, drastically reduced the recall, while only slightly decreasing the build time of the graph.

5.4 VP-Tree Ensemble

For our implementation of the VP-tree ensemble method, the most important parameter to control recall and build time is the number of trees (*n_vp_trees*) that are merged to construct the graph. Figure 5.14 shows for graphs constructed with the ensemble method on the 100K dataset, how the *n_vp_trees* parameter influences build time, search time, and recall. Merging just two VP-tree chunk graphs together is enough to surpass a recall of 70%. When merging more than 6 vantage point trees, a recall of more than 90% can be achieved. The stitching methods we looked at before, did not achieve a recall even close to this, within a similar construction time of 10-15 seconds. As expected, the construction time plot in Figure 5.14, shows a linear relationship between the number of VP-trees, and the total build time. Each additional VP-tree takes the same amount of time as the previous one to be built and merged into the graph.

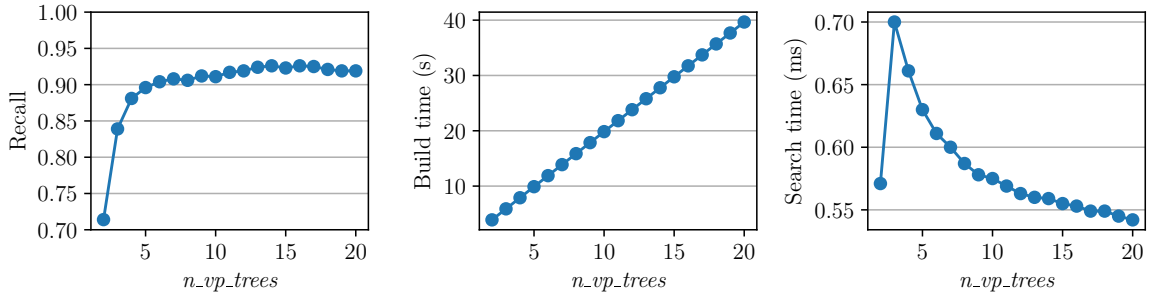


Figure 5.14: Effect of the *n_vp_trees* parameter on recall, search time and build time when constructing a single-layer VP-tree ensemble graph on the 100K dataset. The parameters *m_max* = 40, *same_chunk_m_max* = 16 and *max_chunk_size* = 256 were used for each tree iteration.

The search times on the produced graphs are decreasing with the number of trees, indicating that the graph is slowly optimized over time. While the recall does not improve much after 6-10 trees, the search time still benefits from more iterations. If you compare the search times of the ensemble method in Figure 5.14 with the search times of the stitching methods in Figure 5.12, it becomes apparent that for a fixed construction time, the ensemble method not only achieves better recall but also produces a graph with a lower search time. Since our Rust implementation uses the same memory layout and search functions for the HNSW, stitching, and ensemble methods, the search and build times are highly comparable.

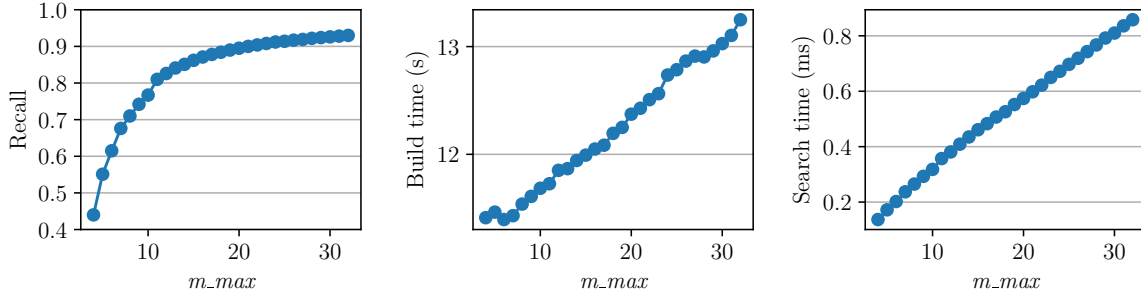


Figure 5.15: Effect of m_max on recall, search time and build time when constructing a single-layer VP-tree ensemble graph on the 100K dataset. Constant parameters are $n_vp_trees = 6$, $max_chunk_size = 256$ and $same_chunk_m_max = m_max$.

As shown in Figure 5.15, the VP-tree ensemble achieves a higher recall if the number of neighbors is increased. However, increasing m_max also linearly increases build times and search times. Regarding its effect on the measured metrics, the m_max parameter is similar to the number of vantage point trees: After a certain point, the recall is only experiencing diminishing returns.

The m_max parameter describes the maximum number of neighbors each point in the graph can have. But we also have the parameter $same_chunk_m_max$ that describes the number of neighbors each point is connected to within its own chunk in one of the VP-trees. The question is if there is any benefit to setting this parameter to a lower value than m_max . Limiting the number of neighbors within a chunk could leave more space for diverse connections across chunk borders. Of course, $same_chunk_m_max$ should never be smaller than $\frac{m_max}{n_vp_trees}$ because then we allocated space for neighbors that are never filled.

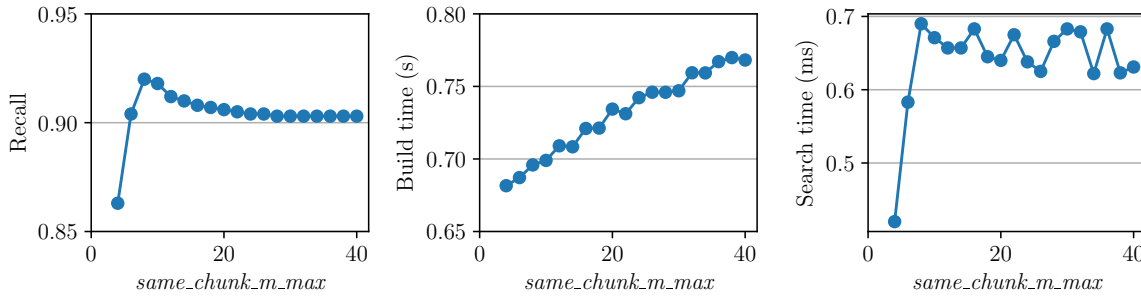


Figure 5.16: Effect of $same_chunk_m_max$ on recall, search time and build time for VP-tree ensembles on the 100K dataset. Constant parameters are $m_max = 40$, $n_vp_trees = 6$ and $max_chunk_size = 256$.

As Figure 5.16 shows, there is a gain in recall when choosing $same_chunk_m_max$ to be around m_max / n_vp_trees . Setting $same_chunk_m_max$ to this fraction means that all connections made within a chunk of any VP-tree, end up in the final graph. Doing so, also results in slightly better build times, because fewer neighbors need to be merged into the

neighbor lists of the global graph. The rightmost graph in Figure 5.16 does not show a conclusive effect for the search times.

Up until now, the chunk stitching and ensemble benchmarks all used VP-trees built by selecting a single random vantage point at every split. However, as discussed before, it is possible to consider $n_candidates$ vantage point candidates at each split and select the one with the greatest distance to the others because it is more in the "corner" of the data space. We put this to the test by building graphs with the stitching and ensemble methods using multiple vantage point candidates. Apart from a slightly increased build time because the candidate selection needs additional computations, we could not find the slightest effect on recall and search times. Therefore, we continue to use VP-trees that use only a single random candidate to select each vantage point. Potentially, the candidate selection scheme is not as meaningful in high-dimensional data. In this sparse space, there might be almost no difference between points that are far or close to one another.

When constructing HNSW indices we saw that the layered nature can cause small speedups in search and build time (See Figure 5.3). Until now, all presented ensemble graphs were just single-layer graphs with the parameter *level_norm* set to 0. If we use greater values for this parameter, a layered structure of multiple stacked graphs is built by the ensemble method. In this case, nodes on higher layers can have m_max neighbors, while nodes on the bottom layer are allowed to have m_max_0 neighbors.

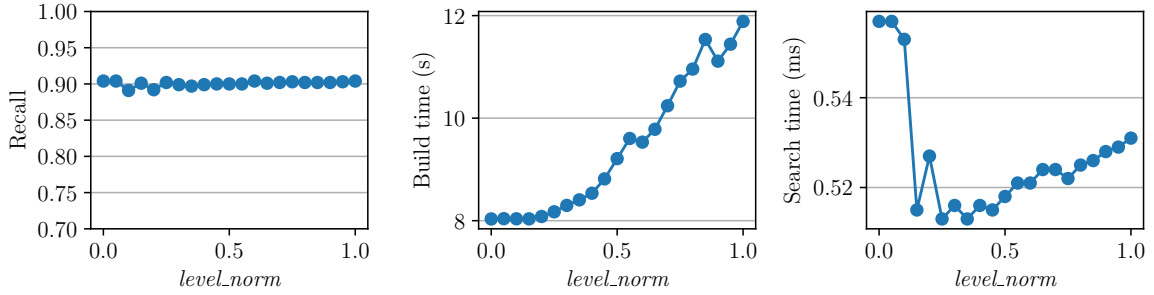


Figure 5.17: Effect of the *level_norm* parameter on the metrics of an HNSW-like structure built by VP-tree ensembles on the 100K dataset. The parameters are *same_chunk_m_max* = m_max = 20 (on higher layers) and m_max_0 = 40 on the bottom layer, n_vp_trees = 6 and *max_chunk_size* = 256.

Figure 5.17 shows that there is virtually no effect on recall when building a layered structure with $level_norm > 0$, but the search times could be slightly better around $level_norm = 0.3 \pm 0.1$. The build times increase if the *level_norm* parameter is raised. Since all layers are built independently of each other, it is not surprising that there is an additional build cost associated with the graph on each layer.

Up to this point, we always used exhaustive distance calculations between all points within a chunk, to insert each point's *same_chunk_m_max* closest neighbors into the graph. This works well for small chunk sizes. But if we want to support a larger chunk size, the number of distance calculations grows quadratically. Therefore, it could be more efficient to use the Relative NN-Descent algorithm to connect nodes within a chunk to each other. This option is integrated into our VP-tree ensemble implementation and gives the user control

over the number of inner and outer iterations.

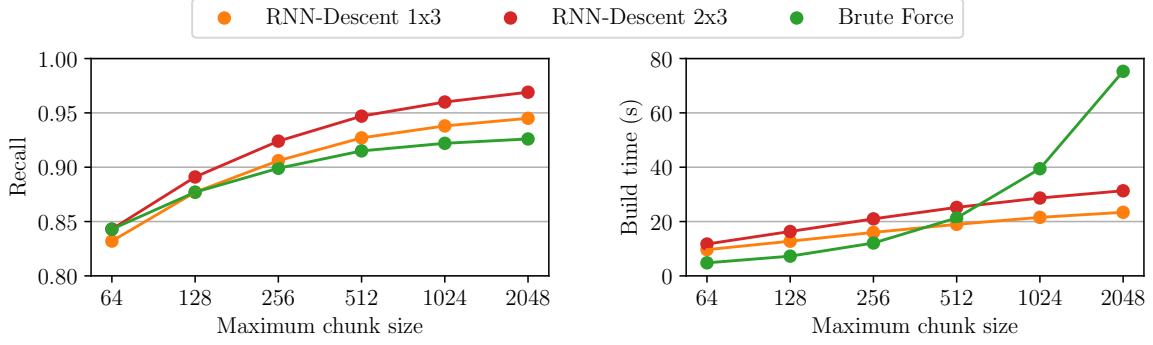


Figure 5.18: Effect of in-chunk point connection strategies on the metrics of VP-tree ensembles. The (single layer) graphs were constructed on the 100K dataset with $n_{vp_trees} = 6$, $m_{max} = 40$ and $same_chunk_m_{max} = 20$. The label *RNN-Descent 1x3* stands for 1 outer, and 3 inner loop iterations, while *RNN-Descent 2x3* uses 2 outer, and 3 inner iterations in the RNN-Descent algorithm.

The *same_chunk_m_max* parameter of the ensemble method dictates the *m_initial* and *m_pruned* parameters of the RNN-Descent algorithm that is run on individual chunks. Figure 5.18 shows the results of experiments, where the efficiency of the brute force method is compared to the RNN-Descent-based one. For a small chunk size of 64 points, the build time of the brute force approach is only half as long as for RNN-Descent, and the recall matches. But as the chunk size increases, the build time required for RNN-Descent grows much more slowly, while also producing graphs that can achieve a higher recall. We found that RNN-Descent with $t_{inner} = 3$ and $t_{outer} = 2$ worked best for a high recall. At least on the 100K dataset, higher iteration counts did not lead to better results.

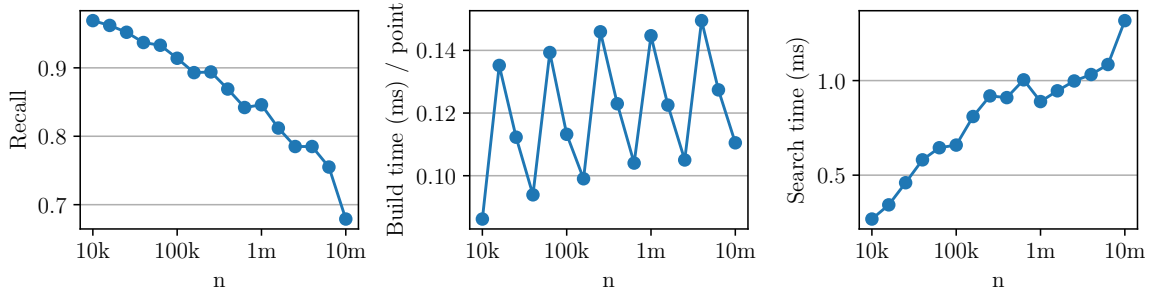


Figure 5.19: Effect of the dataset size n on a single layer graph built by a VP-tree ensemble with $n_{vp_trees} = 6$, $max_chunk_size = 256$, $m_{max} = 40$ and $same_chunk_m_{max} = 10$. Note that the x-axis showing n from 10,000 to 10,000,000 is logarithmic and the build time is reported per point as the total build time divided by n .

Finally, we want to see how the VP-tree ensemble approach scales for larger datasets, compared to an HNSW and RNN-Descent graphs, for which we showed similar experiments in

Figure 5.5 and 5.8. As expected, Figure 5.19 shows that the recall drops and the search time surges for a VP-tree ensemble graph, when n is increased. But what is going on with the periodically fluctuating per-point build time, of the middle plot in Figure 5.19? The fluctuation is an artifact caused by keeping our *max_chunk_size* parameter constant at 256, while the dataset size n is increased 5 times per order of magnitude. The *actual* chunk size is somewhere between *max_chunk_size*/2 and *max_chunk_size*, depending on how often a subtree needs to be split to fall into this range. Because a number of distance calculations quadratic to the *actual* chunk size need to be performed, it influences the build times a lot, explaining the periodic nature of the build times in Figure 5.19.

Table 5.2 shows the numbers of Figure 5.19 in comparison to our HNSW and RNN-Descent experiments, from Figures 5.5 and 5.8. The table compares, how recall, build time, and search time change when n grows by 3 orders of magnitude from $n = 10K$ to $n = 10M$. Compare the values in the *factor* columns across the 3 rows in Table 5.2. We see that the drop in recall associated with larger n , is, unfortunately, stronger for our VP-tree ensemble. However, while the construction of an HNSW is 4.63x as expensive per point when n is increased from 10K to 10M, our VP-tree ensemble has only a 28% increased build time (per point). We suspect that with growing dataset size, the build time differences to the HNSW could grow even larger in favor of our VP-tree ensemble approach.

Table 5.2: Effects of scaling the dataset size from 10K to 10M points for HNSW, Relative NN-Descent and VP-tree ensemble. Parameter details are found in the captions of Figure 5.5, 5.8 and 5.19

Structure	Recall			Build (ms) / point			Search (ms)		
	10K	10M	factor	10K	10M	factor	10K	10M	factor
HNSW	0.971	0.852	0.88x	0.112	0.518	4.63x	0.187	0.709	3.79x
RNN-Descent	0.976	0.894	0.92x	0.182	0.385	2.12x	0.271	1.617	5.97x
VP-tree ensemble	0.969	0.679	0.70x	0.086	0.111	1.28x	0.268	1.317	4.91x

5.5 Benchmarks on the 10M dataset

We benchmarked our HNSW, RNN-Decent, VP-Tree ensemble, and chunk stitching methods on the 10M dataset, provided by the SISAP Challenge Committee [SIS24]. These benchmarks on the 10M dataset should evaluate if the goal of the thesis – speeding up search index construction on large, high-dimensional datasets – was achieved or not.

To assess, how well our index structures fulfill or fail this goal, we set some constraints. A recall of at least 80% needs to be achieved on the 10M dataset for k -NN search with $k = 30$. Since all of our index structures use greedy search, we adjust the *ef* parameter needed to cross the 80% recall threshold for each model individually. The *ef* parameter is a knob that can usually be turned up to gain a higher recall at the cost of longer search times. Another constraint is, that all graphs should store no more than 40 neighbors per point (on each layer, if applicable). These restrictions are arbitrary, but necessary because we do not have the capacity to determine the optimal build parameters for every model type

on the 10M dataset. Limiting the number of neighbors to 40 means that single-layer ensemble and stitching methods use $m_max = 40$, while HSNWs and multi-layer ensembles use $m_max = 20$ and $m_max_0 = 40$. The Relative NN-Descent approach is assigned the parameters $m_initial = m_pruned = 40$.

For RNN-Descent, HNSW, and VP-tree ensemble methods, we developed multithreaded implementations, that try to use all available cores on a given machine to speed up the index construction. Parallel processing is not always available, so we take a look at single-threaded and multithreaded implementations separately. Let us start with evaluating the single-threaded models.

We built a variety of models with different parameters on the 10M dataset and performed approximate k -NN searches with $k = 30$ for the 10K query set with all possible ef values in the set $\{30, 40, 50, \dots, 150\}$. Models that were able to reach at least 80% recall with $ef \leq 150$ are shown as (mostly ascending) lines in the plots of Figure 5.20. Each line represents a single model with a unique parameterization. The lines are colored-coded to signify the type of model, but exact parameters are omitted. The plot should just provide a general idea of how the different model classes behave when a greater ef value is used during search. In general, HSNWs and RNN-Descent graphs seem to have an easier time reaching the minimum recall. The HNSW structures are also a clear winner at achieving low search times. However, the plots also show, that graphs created by the newly proposed VP-tree ensemble method are capable of breaking into the required recall ranges, but sometimes need a higher ef value to get there.

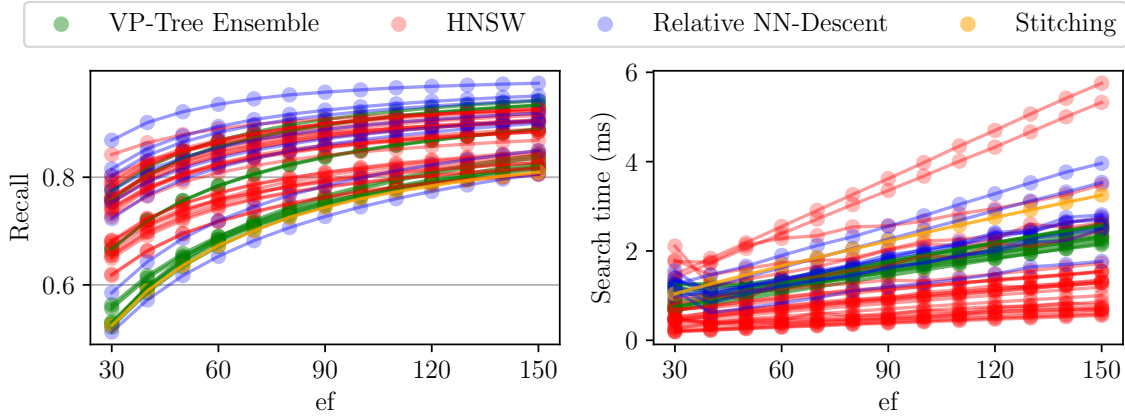


Figure 5.20: Four types of models (HNSW, RNN-Descent, VP-tree ensemble, and chunk stitching) on them 10M dataset and their metrics in relation to the search ef .

We now focus on some concrete parameterizations that are interesting to look at, and given them a unique identifier such that it is easier to refer to them in text.

Based on prior experimentation, our VP-tree ensembles use the parameters $m_max = 40$ and $same_chunk_m_max = 10$. The models *Ensemble6* and *Ensemble10* use 6 and 10 VP-trees, respectively. Their in-chunk connections are done by the brute-force technique with a max_chunk_size of 256 elements. *Ensemble6Rnn* and *Ensemble10Rnn* are two VP-tree ensembles that are also built on 6 and 10 VP-trees, but they use RNN-Descent with $t_outer = 2$ outer loops and $t_inner = 3$ inner loops to connect neighbors within their chunks.

These RNN-Descent ensembles afford a *max_chunk_size* of up to 1024 elements. The ensemble models just described create a flat graph. In contrast, *Ensemble6Layered* builds a multi-level HNSW-like structure. It uses 6 trees and a chunk size of up to 256 points – like *Ensemble6* – but is built with *level_norm* = 0.3, *m_max* = 20, and *m_max_0* = 40 instead. As discussed before, the construction time of the RNN-Descent version of the ensemble method benefits from a low *same_chunk_m_max* parameter, as this means that there is less work to do in terms of updating neighbors of neighbors. The *Ensemble6* and *Ensemble10* methods should not depend on this factor as much, but we keep it at 10 to promote diversity when merging multiple trees.

For the HNSW models, we use *m_max* = 20 and *m_max_0* = 40 as well. We fix the *level_norm* parameter of our HNSW implementation to 0.3, while the other libraries set it automatically to $\frac{1}{\ln(20)} \approx 0.33$. Values of 30, 40, and 60 are used for the *ef_construction* parameter. There are 5 HNSW implementations: our own (*Hnsw*), two others written in Rust (*JpBoth* and *RustCv*), and two created in C++ (*Hnswlib* and *Faiss*). This leaves us with 15 HNSW models to benchmark – *Hnsw30*, *JpBoth30*, ..., *Hnswlib60*, *Faiss60* – named after their library and the *ef_construction* parameter.

Our Relative NN-Descent implementations use *m_initial* = *m_pruned* = 40 for the number of neighbors. *RnnDescent2x3* uses 2 outer loops and 3 inner loops, while *RnnDescent3x3* runs 3 outer and 3 inner loops. Multithreaded construction has been implemented for our VP-tree ensemble, RNN-Descent, and HNSW. This makes it possible to compare single vs. multithreaded performance. The same is true for the HNSW of *Hnswlib* and *JpBoth*. The Rust implementation of an HNSW in *RustCv* is only single-threaded, while the *Faiss* library uses all available cores by default and does not allow the user to run single-threaded index construction.

Experiments on the 10M dataset showed that our chunk stitching approaches did not perform well. All 4 methods struggled to achieve the required 80% recall. We include one model named *Stitching3* in the benchmarks of Table 5.3, but it was worse than the VP-tree ensembles in every aspect. It uses chunk stitching method 3, with *max_chunk_size* = 256, *m_max* = 40 and *x* = 10. A really high fraction parameter of 0.9 was needed to push it across the recall threshold and even then it needs a greedy search with *ef* = 140 and takes longer to search than all of our other methods.

Table 5.3 compares the single-threaded performance of the named models. Next to the build time, the table shows the *ef* search parameter that was needed for the respective model to reach at least 80% recall. The recall and search time columns refer to searches performed with this selected *ef* parameter. The recall time in milliseconds (ms) refers to the average search time for a single query point.

Table 5.3 does not contain a row for the *Faiss* HNSW indices, because they could not be constructed in a single-threaded fashion, so a comparison with the other models would be inadequate. All top 5 ranks of the build time column are occupied by different parameterizations of the VP-tree ensembles. The fastest one is *Ensemble6*, with a graph construction time of only 14m 28s. This is more than twice as fast as *Hnswlib30*, which is with 35m 13s the fastest building HNSW model for the chosen parameters. Our own Rust implementations of an HNSW (*Hnsw30*, *Hnsw40*, and *Hnsw60*) surpass the other two Rust libraries – *JpBoth* and *RustCv* – in terms of build time and search speed.

Table 5.3: Single-threaded build times and search performance on the 10M dataset

Structure	Build time			Rank	ef	Recall	Search time	Rank
<i>Ensemble6</i>	14m	28s		1	130	0.807	2.076ms	14
<i>Ensemble6Layered</i>	14m	51s		2	130	0.802	2.016ms	13
<i>Ensemble10</i>	24m	15s		3	120	0.811	1.842ms	12
<i>Ensemble6Rnn</i>	24m	50s		4	70	0.806	1.191ms	10
<i>Ensemble10Rnn</i>	41m	52s		6	40	0.803	0.888ms	7
<i>RnnDescent2x3</i>	52m	34s		8	110	0.812	2.687ms	16
<i>RnnDescent3x3</i>	1h	9m	25s	11	30	0.815	1.170ms	9
<i>Hnsw30</i>	1h	5m	5s	10	50	0.806	0.565ms	6
<i>Hnswlib30</i>		35m	13s	5	100	0.802	0.430ms	3
<i>JpBoth30</i>	2h	15m	11s	15	140	0.800	2.637ms	15
<i>RustCv30</i>	3h	57m	58s	18	30	0.803	1.391ms	11
<i>Hnsw40</i>	1h	23m	52s	13	40	0.802	0.552ms	5
<i>Hnswlib40</i>		44m	38s	7	60	0.808	0.318ms	2
<i>JpBoth40</i>	2h	37m	49s	16	150	0.692	2.806ms	17
<i>RustCv40</i>	4h	39m	23s	19	150	0.752	4.420ms	19
<i>Hnsw60</i>	1h	49m	54s	14	40	0.822	0.531ms	4
<i>Hnswlib60</i>	1h	3m	55s	9	40	0.826	0.266ms	1
<i>JpBoth60</i>	3h	4m	11s	17	50	0.821	0.913ms	8
<i>RustCv60</i>	6h	10m	44s	20	150	0.795	4.530ms	20
<i>Stitching3</i>	1h	23m	15s	12	140	0.802	3.085ms	18

For all HNSWs, increasing the *ef_construction* parameter led to a higher build time, as expected. But there is a weird effect going on with respect to the recall and search times of the *JpBoth* and *RustCv* HNSW implementations. While our own, and the *Hnswlib* HNSWs achieve better recall and search times with increased *ef_construction*, the *JpBoth30* and *RustCv30* HNSWs achieve better recall and search times than the corresponding *JpBoth40* and *RustCv40*. The latter two do not even cross the 80% recall mark. We currently do not have a plausible explanation for this effect, but it was persistent across many runs and not just a result of random noise.

The recall and required *ef* parameter of our HNSW and the one from *Hnswlib* seem to be comparable. However, the C++ implementation of *Hnswlib* has search times that are almost twice as fast as our HNSW, so further optimizations are needed here.

Interestingly, introducing multiple layers into the VP-tree ensemble approach by using *level_norm* > 0, does not seem to be beneficial. Comparing *Ensemble6* and *Ensemble6Layered* in Table 5.3, we observe a slightly higher build time for the layered graph, while there is virtually no difference between recall (0.807 vs. 0.802) and search times (2.076ms vs. 2.016ms). They also both require an *ef* parameter of at least 130 to cross the 80% recall line. Using more trees like in *Ensemble10* increases the build time linearly, while there are only very small improvements in the recall and search time.

Instead of just using more trees, connecting points within chunks via Relative NN-Descent in the ensemble approach seems to have a profound effect. The RNN-Descent *Ensemble6Rnn* and the brute-force *Ensemble10* models have almost the same construction cost (24m 50s vs. 24m 15s), but *Ensemble6Rnn* needs much less *ef* to reach the recall threshold (70 vs. 120). This also results in a lower search time (1.842ms vs 1.191ms).

It is interesting that the pure RNN-Descent models *RnnDescent2x3* and *RnnDescent3x3* have such a high build time compared to the ensemble models *Ensemble10Rnn* and *Ensemble6Rnn* which use RNN-Descent within their chunks. This is mainly due to the choice of the different parameters regarding the number of neighbors: Even though the ensemble methods also use 2 outer and 3 inner loops, each VP-tree iteration only constructs graphs with 10 neighbors per point (*same_chunk_m_max* = *m_initial* = *m_pruned* = 10). On the other hand, *RnnDescent2x3* and *RnnDescent3x3* need to construct graphs with 40 neighbors, right away. RNN-Descent scales quadratically with the number of neighbors (*m_initial* and *m_pruned*), so e.g. 10 passes of 2x3 RNN-Descent over the dataset, chunked by 1024 elements and using 10 neighbors (*Ensemble10Rnn*), are still faster than performing a single 2x3 pass over the dataset with 40 neighbors per point (*RnnDescent2x3*). The increased memory locality in the chunked version could also be an important factor.

Multithreaded benchmark results for the HNSW, RNN-Descent, and VP-tree ensemble methods are displayed in Table 5.4. HNSWs built by the *Faiss* library are also included here. The indices were built on the 10M dataset, using all 56 threads of our machine.

Table 5.4: Multithreaded build times and search performance on the 10M dataset

Structure	Build time		Speedup	Rank	Recall	Rank	Search time	Rank
<i>Ensemble6</i>	1m	8s	12.6x	1	130	0.802	2.319ms	12
<i>Ensemble6Layered</i>	1m	13s	12.2x	2	140	0.807	2.446ms	13
<i>Ensemble10</i>	1m	52s	13.0x	3	120	0.806	2.029ms	11
<i>Ensemble6Rnn</i>	1m	56s	12.8x	4	70	0.805	1.373ms	9
<i>Ensemble10Rnn</i>	3m	11s	13.1x	5	40	0.805	1.076ms	8
<i>RnnDescent2x3</i>	11m	31s	4.6x	18	150	0.806	3.962ms	16
<i>RnnDescent3x3</i>	14m	26s	4.8x	19	30	0.802	0.960ms	7
<i>Hnsw30</i>	5m	33s	11.7x	8	50	0.805	0.870ms	6
<i>Hnswlib30</i>	4m	21s	8.1x	6	90	0.809	0.541ms	3
<i>JpBoth30</i>	6m	31s	20.7x	10	140	0.801	3.259ms	14
<i>Faiss30</i>	8m	41s	-	14	30	0.825	10.755ms	17
<i>Hnsw40</i>	6m	41s	12.5x	11	40	0.802	0.757ms	5
<i>Hnswlib40</i>	5m	24s	8.2x	7	50	0.803	0.424ms	2
<i>JpBoth40</i>	7m	29s	21.1x	13	150	0.680	3.345ms	15
<i>Faiss40</i>	5m	58s	-	9	40	0.802	11.106ms	18
<i>Hnsw60</i>	9m	2s	12.1x	15	40	0.821	0.732ms	4
<i>Hnswlib60</i>	7m	26s	8.6x	12	40	0.830	0.398ms	1
<i>JpBoth60</i>	9m	6s	20.2x	16	50	0.820	1.606ms	10
<i>Faiss60</i>	10m	18s	-	17	30	0.808	12.649ms	19

Among the multithreaded HNSW implementations in Table 5.4, *Hnswlib* is the clear winner in terms of build and search time. *Faiss* is the slowest HNSW to construct for *ef_construction* parameters of 30 and 60, but *Faiss40* builds almost as fast as *Hnsw40*. The HNSW implementation developed for this thesis still beats *JpBoth* in all aspects, but the difference in build time is much closer than in the single-threaded case. The *JpBoth* HNSW also experiences the greatest speedup in a multithreaded context, being more than 20 times faster than when running only on a single thread. This is likely because the mutex locks used by *JpBoth* for multithreading slow down the single-threaded performance, while our own HNSW implementation does not use any locks in single-threaded mode.

Our Rust implementation still takes about 20% longer to construct the HNSW than the C++ implementation of *Hnswlib*, but its parallelization seems to work well with a speedup factor of around 12x, compared with the single-threaded build times.

One thing that sticks out in Table 5.4, are the very high search times of *Faiss*. It could be, that the *Faiss* library expects queries to be batched and wants to handle them in parallel. To be fair to all implementations however, we pass queries one point at a time, measuring the total time in the wrapping Python code until search results for a query point are returned. It is likely that *Faiss* performs better if not used on a point-by-point basis.

The fastest graph builders in Table 5.4 are again the VP-tree ensemble models. With a build time of only 1m 8s, *Ensemble6* still manages to reach a recall of more than 80%, albeit with a search time ≈ 3 times as long as the search time of our *Hnsw30* model. Much closer to the HNSW are the search times of the *Ensemble10Rnn* model. Here, a pretty good search graph that only needs *ef* = 40 for 80% recall is built in 3m 11s.

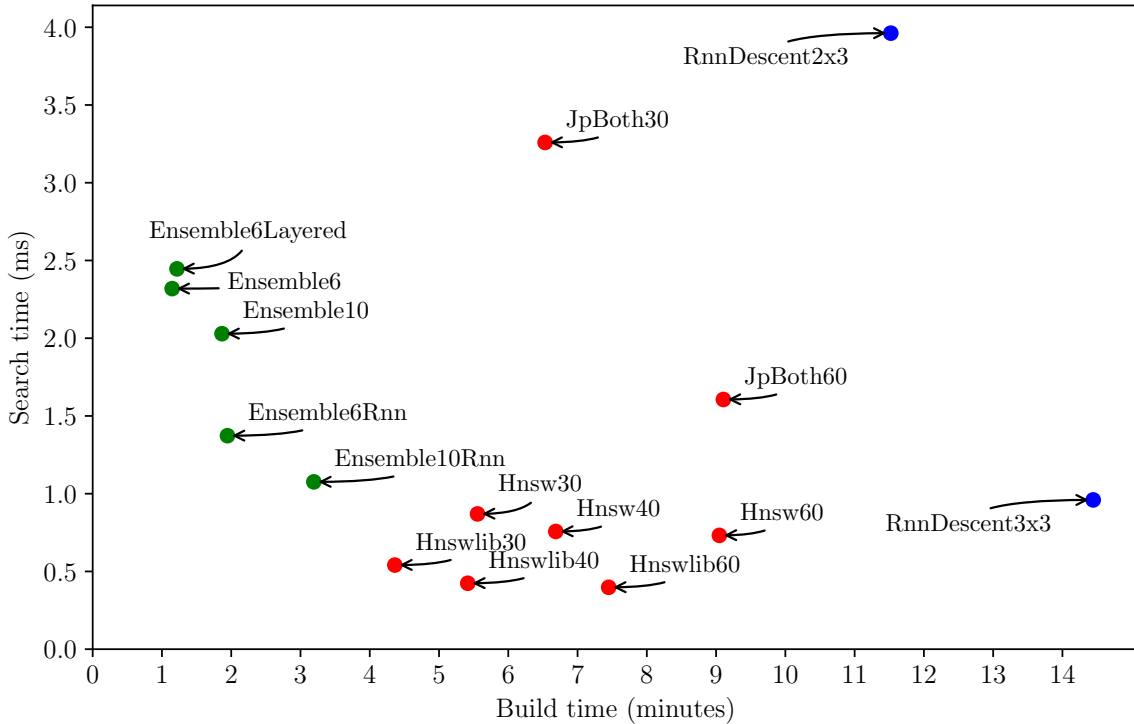


Figure 5.21: Build vs. search time tradeoff: Multithreaded graph structures on the 10M dataset that reach recall > 0.8 with *ef* ≤ 150 and a search time below 5ms.

RNN-Descent alone yields the worst parallelization among the methods in Table 5.4 and the build times are now larger than for the HNSWs. However, the *RnnDescent3x3* model manages to create a graph that is only a few percent points slower in search speed than our HNSW. The RNG-property used in RNN-Descent seems to change the graph structure significantly compared to just choosing the nearest neighbors. This is especially visible in our RNN-Descent enhanced VP-tree ensembles.

Figure 5.21 compares the build and search times of all models that reached at least 80% recall, and a search time of less than 5ms on the 10M dataset. A Pareto front can be thought of to run along points in the bottom left corner of the plot. The models *Ensemble6*, *Ensemble6Rnn*, *Ensemble10Rnn* and all the *Hnswlib* indices are part of this Pareto front. These models could represent reasonable choices in scenarios where a tradeoff between build and search time has to be made. Selecting exactly the point where a model’s recall surpasses 80% leads to a huge variance in the displayed *ef* and search times in Table 5.4. Another perspective can be taken when we conduct greedy search with a candidates list of fixed size $ef = 100$, which is shown in Table 5.5. Here, the variance in recall and search time becomes more apparent. Interestingly, the *RnnDescent3x3* model achieves the highest recall of 92.2%.

Table 5.5: Multithreaded build times, recall, and search time on the 10M dataset for a fixed greedy search $ef = 100$ parameter. The *score* column is the average of the 3 *rank* columns, min-max normalized to a scale between 0 and 1, such that a score of 1 marks the highest (best) rank and 0 the lowest rank.

Structure	Build time		Rank	Recall	Rank	Search time		Score
<i>Ensemble6</i>	1m	8s	1	0.763	16	1.930ms	11	0.74
<i>Ensemble6Layered</i>	1m	13s	2	0.763	16	1.890ms	9	0.77
<i>Ensemble10</i>	1m	52s	3	0.778	14	1.799ms	8	0.83
<i>Ensemble6Rnn</i>	1m	56s	4	0.849	12	1.758ms	7	0.89
<i>Ensemble10Rnn</i>	3m	11s	5	0.907	4	1.897ms	10	1.00
<i>RnnDescent2x3</i>	11m	31s	18	0.745	18	2.789ms	18	0.00
<i>RnnDescent3x3</i>	14m	26s	19	0.922	1	1.970ms	12	0.63
<i>Hnsw30</i>	5m	33s	8	0.863	10	1.207ms	5	0.89
<i>Hnswlib30</i>	4m	21s	6	0.817	13	0.591ms	1	0.97
<i>JpBoth30</i>	6m	31s	10	0.774	15	2.673ms	17	0.34
<i>Faiss30</i>	8m	41s	14	0.920	2	10.341ms	13	0.71
<i>Hnsw40</i>	6m	41s	11	0.885	7	1.242ms	6	0.86
<i>Hnswlib40</i>	5m	24s	7	0.861	11	0.672ms	3	0.94
<i>JpBoth40</i>	7m	29s	13	0.645	19	2.659ms	16	0.17
<i>Faiss40</i>	5m	58s	9	0.882	8	9.485ms	19	0.51
<i>Hnsw60</i>	9m	2s	15	0.907	4	1.205ms	4	0.89
<i>Hnswlib60</i>	7m	26s	12	0.905	6	0.667ms	2	0.97
<i>JpBoth60</i>	9m	6s	16	0.881	9	2.156ms	15	0.40
<i>Faiss60</i>	10m	18s	17	0.913	3	11.123ms	14	0.57

For each model, the sum of its three ranks in build time, search time, and recall is used to compute a *score* column in Table 5.5. The sum of the three ranks was min-max normalized across all models and then inverted, such that a score of 1 signifies the best rank sum and a score of 0 is assigned to the model with the worst rank sum. This ranking score is totally made up, but can still tell the reader at a quick glance which models have some good qualities to them.

The highest score is achieved by *Ensemble10Rnn*, which is characterized by a low build time and a relatively high recall for $ef = 100$. It is surprising that our ensemble approach is able to yield a recall of more than 90% on the 10M dataset while having a quicker build time than all the HNSWs. The constructed graph is also searched slightly faster than the *RnnDescent3x3* graph which has the highest recall overall.

Still, there is a big gap between our own HNSW and the search speed of the *Hnswlib* library by Malkov and Yashunin [MY18]. Any optimizations we would apply to our HNSW to close this gap, will likely directly accelerate searches on the ensemble methods as well, because once built, both methods operate on the same data structure.

We close this chapter by showing two bar plots in Figure 5.22 that summarize the construction time differences between the search indices. On the left side, single-threaded build times are compared, while the bar plot on the right side shows the construction times of the multithreaded models.

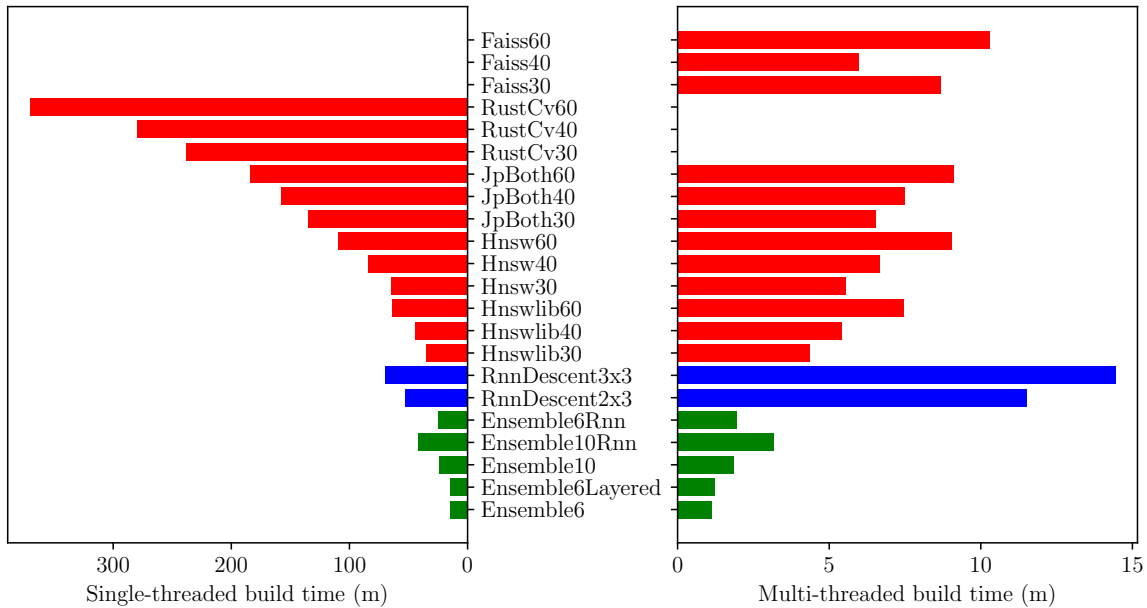


Figure 5.22: Build time comparison between models built on the 10M dataset in a single- and multithreaded context.

6 Conclusion

We explored how the space partitioning properties of a VP-tree can be used to speed up the construction of search graphs. Two proposed methods – *chunk stitching* and *VP-tree ensembles* – were benchmarked on a dataset of high-dimensional embedding vectors, and compared to the existing *HNSW* [MY18] and *RNN-Descent* [OM23] algorithms.

The stitching idea did not work well. It neither achieved a high recall, nor particularly low construction times. However, the VP-tree ensemble approach turned out to be more promising. We have shown that it is able to construct search graphs more quickly than reference implementations of HNSW-indices and RNN-Descent graphs. Especially when the ensemble approach is paired with RNN-Descent to connect local neighbors, we found it to be capable of producing search graphs reaching more than 90% recall on the 768-dimensional dataset of 10 million vectors. The Rust implementation we developed also parallelizes well to multiple threads. Our fastest-building VP-tree ensembles were able to construct a proximity graph on the 10M dataset in only 68 seconds (*Ensemble6*). This is $\approx 3.8\times$ faster than the leading multithreaded HNSW implementation (*Hnswlib30*) by Malkov, Yashunin, et al. [MY+18], while still reaching a recall of more than 80%. At the same time, the resulting graph had $\approx 3.7\times$ longer search times than the just mentioned HNSW. So, the ensemble methods can be effectively used to achieve trade-offs between build and search times that were previously difficult to achieve.

Our Relative NN-Descent implementation was able to create high-quality search graphs but took longer to build than the other methods. The HNSW implementation developed for this thesis was found to outperform two other Rust libraries but still takes $\approx 20\%$ more time to build than an HNSW with the same parameters constructed by *Hnswlib*, and searches are currently almost twice as slow. There is still room for optimizations, which would likely improve the search times of the ensemble methods as well.

Graph construction by VP-tree ensembles has a few advantages over existing methods like HNSW and RNN-Descent. One strength is that it splits up the required computations into many independent units. All trees can be built in parallel; the construction of each VP-tree is parallelized internally; and – after a tree is built – the computations performed on each chunk are independent of each other as well and can be parallelized. To implement a multithreaded HNSW or the RNN-Descent algorithm, access to the neighbor lists of the graphs needs to be synchronized in some way. *Mutexes* – that restrict (mutable) access of a neighbors list to a single thread – are typically used for this, but if many threads need to wait for the same neighbors list to become unlocked again, this could be a performance issue. As measured in our benchmarks, HNSW and RNN-Descent still experience great speedups from parallelism on a single machine, but it is difficult to scale them to a network of independent machines. Our VP-tree ensemble does not have this issue. It could be implemented using the *MapReduce* programming model [DG08] to run on a cluster of machines, but developing that exceeds the scope of this thesis.

Another useful property of the VP-tree ensemble approach is, that the search graph can

be enhanced incrementally. There are scenarios in which a "good enough" search graph is needed in a very short amount of time to execute some early predictions. But over time, more and more iterations can be merged in, to increase the recall for every point in the graph. This is in contrast to e.g. an HNSW, where the final accuracy is mainly determined by initially chosen parameters that cannot be easily changed later. Our VP-tree ensemble is more flexible in that regard, because even additional iterations that use different parameters for *max_chunk_size*, *same_chunk_m_max* or the in-chunk connection strategy can be merged into the main graph. On the other hand, an HNSW makes it very easy to add new nodes to the graph. There is currently no strategy for the VP-tree ensemble to support the insertion of new data.

Another interesting challenge are proximity graphs for data points that are constantly moving. Imagine a large set of ships traveling the oceans, one wants to find the ships closest to some location in case of an emergency. Our VP-tree ensemble could be useful here, if we store more information in the neighbor lists and adjust the merging mechanism. Suppose our compute power allows us to perform a new VP-tree iteration every minute for the set of ships on the oceans, and we want to keep an updated proximity graph. Instead of just replacing the most distant neighbor in the merge step, we could also store the difference in distance between this minute and the last minute. This difference is like a velocity, telling us for every pair of neighbors how quickly they move apart or towards each other. Before merging the next VP-tree ensemble iteration into the global graph, we can update all neighbor lists by approximating the new distances by adding the velocities. This essentially makes the graph into a physics simulation, where every VP-tree iteration updates parts of the neighbor lists, bringing the stored distances and velocities back on track, such that the approximated k -NN graph does not fall out of sync with reality.

Surprisingly, we found in our benchmarks that multi-layer graphs did not provide benefits to the VP-tree ensemble approach. Instead of trying to emulate an HNSW, it might be a better idea to use a quickly built ensemble graph, as a starting point for a Relative NN-Descent pass over the whole dataset. We saw in our benchmarks that the ensembles were the quickest to build, but sometimes lacked in recall. The inverse was true for RNN-Descent. Maybe a VP-tree ensemble can be used to just "jump-start" the Relative NN-Descent algorithm, such that it converges to a high-quality search graph more quickly. Alternatively, the merging step of our VP-tree ensemble could incorporate the RNG-property. We saw in our experiments that choosing *same_chunk_m_max* much lower than *m_max* provided a better recall, so just choosing the closest neighbors does not produce the best search graph (this is the case if *same_chunk_m_max* = *m_max*). However, it is not clear how the RNG-property could be locally enforced during the merge step, without giving up the independence of chunks.

In conclusion, we discovered a new VP-tree ensemble approach in this thesis that builds proximity graphs for k -ANN search on high-dimensional data. On a dataset of 10 million 768-dimensional vectors, it constructed such a graph much faster than other state-of-the-art methods. The ensemble approach is well suited for parallel processing, but it does make some trade-offs in terms of recall and search speed. It would be exciting to conduct further research on the topic in the future because there is a wide range of unexplored parameter combinations, and VP-tree ensembles could probably be used more effectively in conjunction with Relative NN-Descent.

Bibliography

- [Ber08] Daniel J. Bernstein. “ChaCha, a variant of Salsa20”. In: *Technical Report* (2008). URL: <https://cr.yp.to/chacha/chacha-20080128.pdf>.
- [Bey+99] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. “When is “nearest neighbor” meaningful?” In: *Database Theory—ICDT’99: 7th International Conference Jerusalem, Israel, January 10–12, 1999 Proceedings* 7. Springer. 1999, pp. 217–235.
- [BKC08] Marián Boguñá, Dmitri Krioukov, and K. C. Claffy. “Navigability of complex networks”. In: *Nature Physics* 5.1 (Nov. 2008), pp. 74–80. ISSN: 1745-2481. DOI: 10.1038/nphys1130. URL: <http://dx.doi.org/10.1038/NPHYS1130>.
- [Bot24] J. P. Both. *hns-w-rs (Rust library)*. <https://crates.io/crates/hns-w-rs>. 2024.
- [DG08] Jeffrey Dean and Sanjay Ghemawat. “MapReduce: simplified data processing on large clusters”. In: *Communications of the ACM* 51.1 (2008), pp. 107–113.
- [DLM24] DLMF. *NIST Digital Library of Mathematical Functions*. <https://dlmf.nist.gov/>, Release 1.2.2 of 2024-09-15. F. W. J. Olver, A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain, eds. 2024. URL: <https://dlmf.nist.gov/>.
- [DML11] Wei Dong, Charikar Moses, and Kai Li. “Efficient k-nearest neighbor graph construction for generic similarity measures”. In: *Proceedings of the 20th international conference on World wide web*. 2011, pp. 577–586.
- [Dou+19] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvassy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. *The Faiss library*. 2019. arXiv: 2401.08281 [cs.LG]. URL: <https://arxiv.org/abs/2401.08281>.
- [HKP11] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Elsevier Science & Technology, 2011.
- [Hoa61] C. A. R. Hoare. “Algorithm 65: find”. In: *Commun. ACM* 4.7 (July 1961), pp. 321–322. ISSN: 0001-0782. DOI: 10.1145/366622.366647. URL: <https://doi.org/10.1145/366622.366647>.
- [Hoa62] C. A. R. Hoare. “Quicksort”. In: *The Computer Journal* 5.1 (Jan. 1962), pp. 10–16. ISSN: 0010-4620. DOI: 10.1093/comjnl/5.1.10. eprint: <https://academic.oup.com/comjnl/article-pdf/5/1/10/1111445/050010.pdf>. URL: <https://doi.org/10.1093/comjnl/5.1.10>.
- [IEE08] IEEE. “IEEE Standard for Floating-Point Arithmetic”. In: *IEEE Std 754-2008* (2008), pp. 1–70. DOI: 10.1109/IEEESTD.2008.4610935.

- [Jam+13] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning*. Vol. 112. Springer, 2013.
- [Kle00] Jon M Kleinberg. “Navigation in a small world”. In: *Nature* 406.6798 (2000), pp. 845–845.
- [Lec+98] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: [10.1109/5.726791](https://doi.org/10.1109/5.726791).
- [Mal+12] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. “Scalable Distributed Algorithm for Approximate Nearest Neighbor Search Problem in High Dimensional General Metric Spaces”. In: *Similarity Search and Applications*. Ed. by Gonzalo Navarro and Vladimir Pestov. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 132–147. ISBN: 978-3-642-32153-5.
- [Mat15] Niko Matsakis. *Rayon: Data Parallelism in Rust*. <https://smallcultfollowing.com/babysteps/blog/2015/12/18/rayon-data-parallelism-in-rust/>. 2015. URL: <https://github.com/rayon-rs/rayon>.
- [Mil67] Stanley Milgram. “The small world problem”. In: *Psychology today* 2.1 (1967), pp. 60–67.
- [MK14] Nicholas D Matsakis and Felix S Klock II. “The rust language”. In: *ACM SIGAda Ada Letters*. Vol. 34. 3. ACM. 2014, pp. 103–104.
- [MM19] Luke Mathieson and Pablo Moscato. “An Introduction to Proximity Graphs”. In: *Business and Consumer Analytics: New Ideas*. Ed. by Pablo Moscato and Natalie Jane de Vries. Cham: Springer International Publishing, 2019, pp. 213–233. ISBN: 978-3-030-06222-4. DOI: [10.1007/978-3-030-06222-4_4](https://doi.org/10.1007/978-3-030-06222-4_4). URL: https://doi.org/10.1007/978-3-030-06222-4_4.
- [MY+18] Yu. A. Malkov, D. A. Yashunin, et al. *Hnswlib - fast approximate nearest neighbor search*. <https://github.com/nmslib/hnswlib>. Version 0.8.0. 2018.
- [MY18] Yu. A. Malkov and D. A. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2018. arXiv: [1603.09320](https://arxiv.org/abs/1603.09320) [cs.DS]. URL: <https://arxiv.org/abs/1603.09320>.
- [OLV18] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. “Representation learning with contrastive predictive coding”. In: *arXiv preprint arXiv:1807.03748* (2018).
- [OM23] Naoki Ono and Yusuke Matsui. *Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search*. 2023. arXiv: [2310.20419](https://arxiv.org/abs/2310.20419) [cs.IR]. URL: <https://arxiv.org/abs/2310.20419>.
- [Pug90] William Pugh. “Skip lists: a probabilistic alternative to balanced trees”. In: *Communications of the ACM* 33.6 (1990), pp. 668–676.
- [Rad+21] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: [2103.00020](https://arxiv.org/abs/2103.00020) [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.

- [Sam84] Hanan Samet. “The Quadtree and Related Hierarchical Data Structures”. In: *ACM Comput. Surv.* 16.2 (June 1984), pp. 187–260. ISSN: 0360-0300. DOI: [10.1145/356924.356930](https://doi.org/10.1145/356924.356930). URL: <https://doi.org/10.1145/356924.356930>.
- [Sch+22] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. *LAION-5B: An open large-scale dataset for training next generation image-text models*. 2022. arXiv: [2210.08402](https://arxiv.org/abs/2210.08402) [cs.CV]. URL: <https://arxiv.org/abs/2210.08402>.
- [SIS24] SISAP Challenge Committee. *SISAP Indexing Challenge 2024: LAION2B and Projections*. Website. Accessed: 2024-08-05. 2024. URL: <https://sisap-challenges.github.io/2024/datasets/>.
- [TAC23] Eric S. Tellez, Martin Aumüller, and Edgar Chavez. “Overview of the SISAP 2023 Indexing Challenge”. In: *Similarity Search and Applications*. Ed. by Oscar Pedreira and Vladimir Estivill-Castro. Cham: Springer Nature Switzerland, 2023, pp. 255–264. ISBN: 978-3-031-46994-7.
- [Tou80] Godfried T. Toussaint. “The relative neighbourhood graph of a finite planar set”. In: *Pattern Recognition* 12.4 (1980), pp. 261–268. ISSN: 0031-3203. DOI: [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7).
- [Vas+23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [VCC98] Baba C. Vemuri, Y Cao, and Li Chen. “Fast collision detection algorithms with applications to particle flow”. In: *Computer Graphics Forum*. Vol. 17. 2. Wiley Online Library. 1998, pp. 121–134.
- [Wan+13] Jingdong Wang, Jing Wang, Gang Zeng, Rui Gan, Shipeng Li, and Baining Guo. *Fast Neighborhood Graph Search using Cartesian Concatenation*. 2013. arXiv: [1312.3062](https://arxiv.org/abs/1312.3062) [cs.CV]. URL: <https://arxiv.org/abs/1312.3062>.
- [WHS17] Zhiguo Wang, Wael Hamza, and Linfeng Song. *k-Nearest Neighbor Augmented Neural Networks for Text Classification*. 2017. arXiv: [1708.07863](https://arxiv.org/abs/1708.07863) [cs.CL]. URL: <https://arxiv.org/abs/1708.07863>.
- [Wor21] Geordon Worley. *hnsw (Rust library)*. <https://crates.io/crates/hnsw/>. Version 0.11.0. 2021.
- [Yia93] Peter N. Yianilos. “Data structures and algorithms for nearest neighbor search in general metric spaces”. In: *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '93. Austin, Texas, USA: Society for Industrial and Applied Mathematics, 1993, pp. 311–321. ISBN: 0898713137.